



كلية الهندسة بنات

Faculty of Engineering Girls



قسم هندسة النظم والحاسبات

Sys. & Comp. Eng. Dep.



جامعة الازهر

Al-Azhar University

Graduation project

SDV with ADAS For Next Generation Automotive Application

**A Project Submitted in Partial Fulfillment
of the Requirements for the Degree of Bachelor of Science
in Systems and Computers Engineering**

Submitted by

1- Amal Osama Abd El-razk Hazaa	20258240
2- Rehab Ahmed Hassan Ali	20258293
3- Samar Gamal Mahmoud Eid	20258327
4- Shereen Alaa Mohamed Ali	20258335

Supervised by

Ass., Prof., Khaled Elshafey

2025

Table of contents

Chapter 1 : Introduction	7
1.1 Background and Motivation	7
1.2 Advanced Driver Assistance Systems (ADAS) Overview	7
1.2.1 Common Categories of ADAS Features	8
1.3 Software Defined Vehicle overview	9
1.3.1 Software Defined Vehicle concept	9
I. Key Characteristics of SDVs	9
1.4 Problem Statement	10
1.5 Project Objectives	11
1.6 Project Scope	11
1.7 Out of Scope	12
1.8 System Overview	13
1.9 Technical Approach	13
1.10 Infotainment System	13
Chapter 2: Related Work and Survey on Software-Defined Vehicles (SDVs)	14
2.1 From Legacy ECUs to Software-Defined Vehicles	14
2.2 Architecture of Software-Defined Vehicles	15
2.3 Over-the-Air Updates and Modular Design	17
2.4 Integration of ADAS Functions in SDV Frameworks	19
2.5 In-Vehicle Communication and Real-Time Messaging	20
Chapter 3 : Analysis, Tools, and Design	23
3.1 Analysis	23
3.1.2 Goals and Objectives	23
3.2 Tools and Platforms	23
3.2.1 Hardware Platforms	23
3.2.2 Software Platforms	23
3.2.3 Development Tools	24
3.3 System Design	24
3.3.1 Architectural Overview	24
3.3.2 ADAS Module Design	24
3.3.3 Containerization Strategy	25
3.3.4 Infotainment Interface	25

3.4 Security & Maintenance	25
3.5 Summary 25	25
CHAPTER 4: Implementation	26
4.1 ACTUATION AND PERCEPTION LAYERS	26
4.1.1 Introduction	26
4.1.2 System Architecture Overview	26
4.1.3 Perception Layer	27
4.1.4 Actuation Layer	32
4.1.4 SPI Communication Layer	34
4.1.5 Conclusion	36
4.2 Communication Module - SPI-MQTT Gateway	37
4.2.1 Introduction	37
4.2.2 Module Architecture and Implementation	38
4.2.3 Summary	42
4.3 EMERGENCY BRAKING SYSTEM	43
4.3.3 Enable/Disable Mechanism	45
4.3.4 Flowchart	46
4.3.5 Conclusion and Future Work	47
4.4 Blind Spot Detection	48
4.4.1 Introduction	48
4.4.2 Module Architecture and Implementation	48
4.4.1 Introduction	48
4.4.2 Module Architecture and Implementation	48
4.4.3 Enable/Disable Mechanism	51
4.4.4 Flowchart	52
4.4.5 Conclusion and Future Work	53
4.5 Traffic Sign and Traffic Light Recognition System	54
4.5.1 Introduction	54
4.5.2 System Architecture Overview	54
4.5.3 Object Detection Model Implementation	54
4.5.4 Model Conversion: PyTorch to ONNX	56
4.5.5 Classification System	57
4.5.6 Communication Protocol: MQTT Integration	58

4.5.7 Enhanced Frame Comparison Algorithm	59
4.5.5 Camera Control System	60
4.5.9 Future Enhancements	61
4.5.10 Conclusion	61
4.6: Linux Image	62
4.6.1 Overview	62
4.6.2 Embedded Linux	62
4.6.3 Yocto Project	62
4.6.4 Containers in Automotive Software	63
4.6.5 Eclipse Leda	64
4.6.2: Over-the-Air (OTA) Image Update Workflow for Embedded Systems	67
4.6.7 Testing and Validation of Containers	69
4.6.8 Security Considerations	69
4.6.9 Summary.....	70
4.7: INFOTAINMENT SYSTEM.....	70
4.7: INFOTAINMENT SYSTEM.....	70
4.7.1 Introduction.....	70
4.7.2 System Design.....	72
4.7.3 Implementation.....	74
4.7.4 GUI Implementation.....	76
4.7.5 Summary 79.....	79
4.7.6 Future Work	79
CHAPTER 5: Experimental Extension: Lane Keeping Assist (LKA).....	80
5.1 Introduction.....	80
5.2 System Architecture Overview.....	80
5.3 Computer Vision Implementation.....	80
5.3.1 Image Preprocessing Pipeline.....	81
5.3.2 Region of Interest (ROI) Definition.....	82
5.4. Lane Detection Algorithm.....	83
5.4.1 Hough Line Transform Implementation.....	83
5.5. Steering Guidance System.....	83
5.5.1 Steering Angle Calculation.....	84
5.5.2 Direction Classification.....	85

5.6. Communication Protocol: MQTT Integration.....	85
5.6.1 MQTT Architecture.....	85
5.6.2 Command Processing.....	85
5.6.3 Data Transmission Optimization.....	86
5.7. Safety and Reliability Features.....	86
5.8. Integration Capabilities.....	87
5.9. Future Enhancements.....	87
5.9.1 Planned Improvements.....	87
5.9.2 Technology Integration.....	88
5.10. Conclusion.....	88
CHAPTER 6: Conclusion And Future Work.....	89
Conclusion.....	89
Future Work.....	90
REFERENCES.....	91

Table of figures

figure 1.8 Block diagram showing system hardware architecture.....	10
Figure 1.1: Block diagram showing the hardware architecture and communication between system components	12
Table 2.1: Comparison of Legacy ECU Architecture vs. Software-Defined Vehicle Architecture (SDV).....	16
Figure 4.1.1: Perception and Actuation Layers Architecture.....	27
Figure 4.1.2: perception layer flowchart overview.....	30
Figure 4.1.3: perception layer flowchart details.....	31
Figure 4.1.4 actuation layer flowchart.....	33
Figure 4.2.1: Block diagram of the Communication module within the Software-Defined vehicle architecture.....	41
Figure 4.3.1: UML class diagram showing the internal structure of the Emergency Braking Software module.....	44
Figure 4.4.2: Block diagram of the Blind Spot Detection (BSD) module within the Software-Defined vehicle architecture.....	51
Figure 4.4.3: Flowchart representing the internal logic of the Blind Spot Detection (BSD).....	52
Figure 4.5.1: Flowchart representing the internal logic of the Traffic Sign Recognition (TSR).....	54
Figure 4.5.2: YOLO Versions' Performance Metrics	55
Figure 4.5.3: Analysis of Loss Functions and Recall Metrics.....	56
Figure 4.5.4: Model Output.....	56
Figure 4.6.1: Over-the-Air (OTA) Container Update Workflow for Embedded Systems.....	66
4.6.2: Over-the-Air (OTA) Image Update Workflow for Embedded Systems.....	67
Figure 4.7.3: ADAS Container Architecture and Communication Workflow in Eclipse Leda.....	69
Figure 4.7.1: Infotainment System block diagram.....	72
Figure 4.7.2: Data flow within the system.....	73
Figure 4.7.3: Infotainment system control flow.....	75
Figure 4.7.4: Start Window of the infotainment system.....	76
Figure 4.7.5: Main Window displaying ADAS notifications.....	77
Figure 4.7.6: Main Window with sliding settings panel open.....	78
Figure 5.2: Visualizing Image Processing Techniques: Filters and Edge Detection	82
Figure 5.3: Visualizing ROI (region of interest)	83
Figure 5.4: Simulate lane-keeping assistance using adaptive model predictive controller	84

CHAPTER 1: Introduction

1.1 Background and Motivation

The automotive industry is undergoing a transformative shift with the rise of Software Defined Vehicles (SDVs) vehicles in which features and functions are increasingly determined by software rather than hardware alone. SDVs enable faster updates, over-the-air functionality, and adaptability to evolving driving needs and safety protocols. One of the most critical domains where this transformation is evident is in the field of Advanced Driver Assistance Systems (ADAS), which aim to improve vehicle safety and enhance driving comfort by automating, adapting, or enhancing aspects of the driving experience.

In This project, we set out to develop a scaled prototype that showcases the fundamental architecture and functional features of SDVs, focusing on selected ADAS technologies. Our prototype demonstrates the layered software-hardware integration commonly seen in modern intelligent vehicles. By building an environment where sensors feed real-time data into a decision-making system that then communicates with control hardware, the project achieves the goal of simulating a semi-autonomous driving experience.

Our motivation stems from the increasing global emphasis on road safety, autonomous systems, and smart mobility. With the development of artificial intelligence, embedded systems, and affordable computing power, the future of automotive technology is leaning heavily on software-driven intelligence. Countries are mandating ADAS features such as automatic braking, lane assistance, and driver monitoring in new vehicles. As students in systems and computer engineering, we aim to explore the technical foundations of these trends.

1.2 Advanced Driver Assistance Systems (ADAS) Overview

Advanced Driver Assistance Systems (ADAS) are electronic systems installed in vehicles to assist drivers in driving and parking functions. Leveraging a combination of sensors, cameras, and embedded control units, ADAS technologies aim to improve road safety, reduce human error, and enhance the overall driving experience. These systems represent a foundational step toward the development of fully autonomous vehicles and are becoming increasingly mandatory in modern car manufacturing.

ADAS functionalities vary in complexity — from simple warning systems (e.g., lane departure alert, blind spot notification) to fully automated interventions (e.g., emergency braking, adaptive cruise control). They continuously monitor the environment around the vehicle, interpret sensory inputs, and either warn the driver or take proactive action to prevent accidents.

1.2.1 Common Categories of ADAS Features

- Information and Warning Systems:
 - Blind Spot Monitoring (BSM)
 - Lane Departure Warning (LDW)
 - Traffic Sign Recognition (TSR)
 - Forward Collision Warning (FCW)
- Control and Intervention Systems:
 - Automatic Emergency Braking (AEB)
 - Adaptive Cruise Control (ACC)
 - Lane Keeping Assist (LKA)
 - Parking Assistance
- Driver Status Monitoring:
 - Drowsiness Detection
 - Driver Attention Monitoring

1.3 Software Defined Vehicle overview

1.3.1 Software Defined Vehicle concept

The term Software Defined Vehicle (SDV) refers to a new generation of vehicles where most of the core functionalities are controlled and managed by software rather than by fixed hardware logic. Unlike traditional vehicles, where each function (e.g., braking, steering, infotainment) was often tied to dedicated electronic control units (ECUs), an SDV centralizes and abstracts these capabilities using a unified software platform. This results in a vehicle that is more modular, upgradable, and adaptable to new technologies and user preferences.

I. Key Characteristics of SDVs

- **Software-Centric Architecture:**
SDVs decouple vehicle functionality from hardware. New features can be introduced or enhanced via software updates without changing the physical components of the vehicle.
- **Over-the-Air (OTA) Updates:**
Much like smartphones, SDVs can receive remote updates to fix bugs, enhance performance, or introduce entirely new features — such as improved ADAS logic, user interface changes, or battery optimization algorithms.
- **Centralized Computing Platforms:**
SDVs rely on centralized high-performance computing units that consolidate the processing of multiple sensor inputs and vehicle functions — replacing the distributed control architecture of legacy vehicles.
- **Cloud Integration & Connectivity:**
SDVs are typically connected to cloud services that provide data storage, machine learning model updates, fleet management, predictive maintenance, and even remote diagnostics.
- **Personalization & Flexibility:**
Vehicle behaviors can be adapted to the preferences of different drivers or driving contexts. For example, a user could choose between aggressive or

eco-friendly driving modes, or tailor ADAS features to specific comfort levels.

II. Benefits of SDVs:

- Faster Innovation Cycles:

Vehicle manufacturers can deploy new features or fix issues post-production.

- Cost Efficiency:

Reduces the need for specialized hardware modules, leading to reduced manufacturing complexity.

- Enhanced User Experience:

With the help of infotainment systems and cloud services, SDVs offer personalized, intuitive, and feature-rich driving experiences.

- Improved Safety & Autonomy:

ADAS and autonomous driving functions benefit from centralized processing and sensor fusion, allowing more advanced decision-making.

III. SDVs vs. Traditional Vehicles

Feature	Traditional Vehicle	Software Defined Vehicle
Feature Updates	Requires hardware changes	OTA software updates
Control Systems	Many dedicated ECUs	Centralized computing
Sensor Data Handling	Isolated processing	global view
ADAS Capabilities	Basic, fixed at manufacture	Flexible, upgradable
User Interaction	Static controls	app-based interfaces

Table 1.1 SDVs vs Traditional Vehicles

1.4 Problem Statement

Modern vehicles are becoming increasingly intelligent, integrating advanced software systems to enhance driver safety and automate routine tasks. However, in many educational or low-cost embedded system projects, Advanced Driver Assistance Systems (ADAS) are often implemented in isolation, without reflecting the layered and modular nature of real automotive systems. These systems typically lack a scalable architecture where perception, decision-making, and actuation are logically separated and handled by distinct hardware or software modules.

Additionally, while industry standards push toward centralized software platforms and over-the-air feature delivery, many prototypes do not reflect this software-defined paradigm. There exists a gap between academic ADAS demos and real-world Software Defined Vehicle (SDV) implementations, particularly in how features are activated, controlled, and monitored via software interfaces. Therefore, there's a need for a compact, modular, and software-centric prototype that accurately models the SDV architecture and integrates multiple ADAS features with user-configurable behavior.

1.5 Project Objectives

The main objective of this project is to implement a miniaturized Software Defined Vehicle prototype that integrates three key ADAS features — Traffic Sign Recognition (TSR), Blind Spot Detection (BSD), and Emergency Braking System (EBS). We also aimed to develop a user interface (infotainment system) that allows monitoring and controlling these features.

1.6 Project Scope

This project focuses on designing and implementing a prototype that models the architecture of a Software Defined Vehicle (SDV) integrated with selected Advanced Driver Assistance Systems (ADAS). The scope of the project includes:

- Implementation of Three ADAS Features:
 - Traffic Sign Recognition (TSR) using Raspberry Pi Camera and YOLO object detection.
 - Blind Spot Detection (BSD) using two side-mounted ultrasonic sensors.
 - Emergency Braking System (EBS) using a front-facing ultrasonic sensor.
- Hardware Integration:
 - STM32F401RC microcontroller for perception and control (ultrasonic interface and actuator handling).
 - Raspberry Pi 4 for decision-making, ADAS logic, image processing, and user interface.

- Software Architecture:
 - FreeRTOS for real-time task management on STM32.
 - SPI-based communication protocol between STM32 and Raspberry Pi.
 - Python-based ADAS controller and infotainment system.

1.7 Out of Scope:

- Implementation of radar, LiDAR, or high-speed automotive buses (CAN, FlexRay).
- Full-scale autonomous driving or dynamic vehicle control.
- V2X communication.

1.8 System Overview

The system comprises three layers: the perception layer handled by the STM32F401RC microcontroller, the decision-making layer implemented using a Raspberry Pi 4, and the control layer that drives actuators based on the interpreted sensor data. Communication between layers is achieved using SPI protocols, and tasks on STM32 are managed using FreeRTOS for real-time operation.

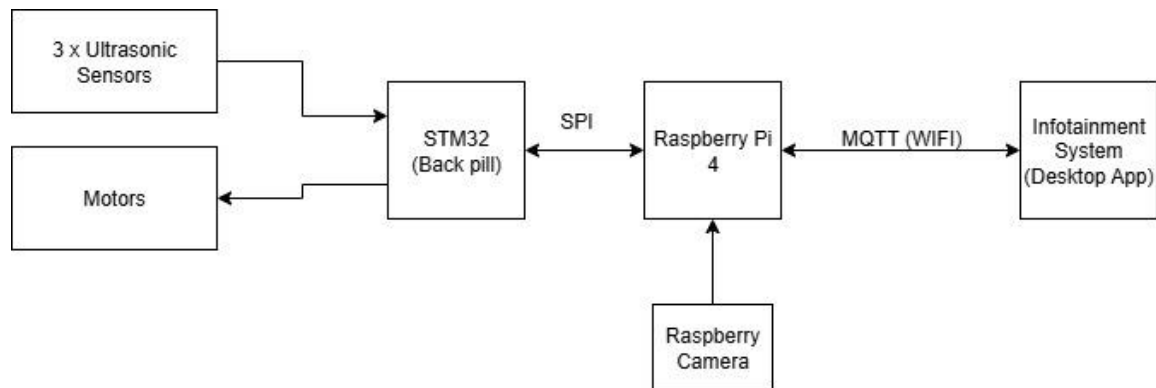


Figure 1.1: Block diagram showing the hardware architecture and communication between system components.

1.9 Technical Approach

For TSR, we used the Raspberry Pi camera in conjunction with a YOLO-based object detection model (You Only Look Once) to detect and classify traffic signs in real-time. YOLO offers both speed and accuracy, making it suitable for embedded systems like the Raspberry Pi. We trained the YOLO model on a dataset of common traffic signs and deployed it within our ADAS application running on the Pi. This allowed for responsive recognition of regulatory and warning signs.

BSD and EBS features were implemented using ultrasonic sensors connected to the STM32F401RC microcontroller. These sensors monitor the environment in blind spots and directly in front of the vehicle. Data collected is sent to the Raspberry Pi, which analyzes the readings and issues control commands when necessary. STM32 then actuates braking mechanisms or alerts based on these decisions.

1.10 Infotainment System

We developed an infotainment GUI on the Raspberry Pi that allows the user to enable or disable any ADAS feature, choose between automatic action or notification-only modes, and visualize sensor data and system states. This layer simulates the human-vehicle interaction commonly seen in modern cars.

CHAPTER 2: Related Work and Survey on Software-Defined Vehicles (SDVs)

2.1 From Legacy ECUs to Software-Defined Vehicles

Automotive electronics have evolved from a multitude of standalone electronic control units (ECUs) toward a more centralized, software-centric paradigm. In a traditional vehicle, dozens of ECUs are each dedicated to specific functions (engine control, braking, infotainment, etc.), with software tightly integrated to hardware. This legacy distributed architecture, while effective for isolated functions, has led to **increasing complexity** as modern cars now host up to 100 or more ECUs. Managing so many units creates challenges in wiring, integration, and updates. Manufacturers are therefore moving away from this **federated ECU design** and consolidating computing power into **domain controllers** and central platforms – the foundation of the *Software-Defined Vehicle (SDV)* concept.

An SDV is defined as a vehicle whose functionality and features are enabled primarily through software, rather than through dedicated hardware alone. In essence, an SDV can **manage operations, add new capabilities, and even fix bugs via software updates** delivered over its lifetime. Deloitte aptly describes this shift as *“the gradual transformation of automobiles from highly electromechanical machines to intelligent, expandable electronic terminals that can be continuously upgraded”*. This paradigm allows a car to evolve after production – **improving performance, adding features, and enhancing safety through code** – something not possible in hardware-driven designs of the past. Advanced features like autonomous driving and ADAS (Advanced Driver-Assistance Systems) have further spurred this evolution. Modern ADAS capabilities (e.g. lane keeping, collision avoidance) demand immense processing and coordination, which legacy architectures struggle to support. Industry trends and research consistently show that **autonomous driving (AD) and ADAS requirements are driving designers to shift from traditional ECU-based systems to domain and central compute units**.

Why transition to centralized software-defined architectures? Several key factors motivate this shift. First, maintaining dozens of ECUs makes it difficult to update software or patch issues. Consolidating functions onto a few powerful processors facilitates **secure over-the-air (OTA) updates** – reducing recall and maintenance costs by allowing remote updates and even rollbacks if needed. Second, a centralized architecture promotes **modularity and reuse**: hardware and software can be developed in a more **collaborative, modular fashion**, speeding up innovation and time-to-market. Third, modern system-on-chip (SoC) technology enables **silicon integration**, packing

what used to be multiple controllers into one chip. High-performance automotive processors now include multi-core CPUs, GPUs, NPUs (neural processing units), and accelerators, all in a single package. This consolidation yields better power efficiency and computational capability for complex applications like AI-based ADAS. Finally, reducing the number of ECUs (and locating a few controllers in vehicle “zones”) **dramatically cuts down wiring harness complexity and weight**. In conventional designs, wiring can account for a significant portion of cost and weight; zonal architectures (grouping devices by region of the car) shorten cables and simplify production. Together, these factors – OTA updatability, modular design, integrated high-performance silicon, and streamlined wiring – underpin the industry’s move toward the software-defined vehicle.

2.2 Architecture of Software-Defined Vehicles

In an SDV, the vehicle’s E/E (electric/electronic) architecture is built to be flexible and software centric. Rather than isolated ECUs communicating over slow buses, SDVs typically use **centralized or zonal computing units** connected via high-speed networks. Table 2.1 summarizes key differences between traditional ECU-based design and modern SDV architecture:

Aspect	Traditional ECU-Based Design	Software-Defined Vehicle Architecture
Compute Architecture	Dozens of discrete ECUs each dedicated to a specific function.	Fewer, high-performance controllers (domain or zonal ECUs) running many functions in software. Central or zonal compute units consolidate tasks.
Software/Hardware Coupling	Software tightly integrated with hardware; changes often require hardware mods.	Abstraction layers decouple software from hardware. Applications run on a common platform (often an embedded OS), allowing hardware changes without rewriting software.
Feature Updates & Upgrades	New features rarely added post-sale; updates require dealership visits or new hardware.	Over-the-air (OTA) updates enable continuous feature upgrades and bug fixes remotely. Vehicle functionality can improve over time via software.
ADAS Integration	ADAS features (e.g. blind-spot detection, emergency braking) implemented on separate ECUs or added modules.	ADAS algorithms integrated into central compute. Shared sensor data and AI models run on centralized processors; ADAS functions are updatable software modules.

Communication	Primarily CAN/FlexRay networks with point-to-point signals; limited bandwidth.	High-speed Ethernet backbones with service-oriented protocols (SOME/IP, DDS) for data exchange. Publish/subscribe messaging (e.g. MQTT) used for asynchronous data and cloud connectivity.
Development & Deployment	Long, rigid development cycles; software updates require extensive re-validation.	Agile, modular development with containerized services and continuous integration. New apps can be deployed in containers or sandboxes on the vehicle OS, accelerating innovation.

Table 2.1: Comparison of Legacy ECU Architecture vs. Software-Defined Vehicle Architecture (SDV).

One hallmark of SDV design is the adoption of powerful centralized computing platforms. For instance, some automakers are introducing **central domain controllers** that handle previously distributed tasks (powertrain, ADAS, infotainment, etc.) using multi-core SoCs. BMW’s upcoming *Neue Klasse* platform, for example, will host *all ADAS functions on a single high-performance domain controller* (with a Snapdragon automotive SoC) rather than separate ADAS ECUs. This centralization not only improves performance (enabling richer sensor fusion and faster decisions), but also simplifies integration – the ADAS software components share a common memory and data environment, rather than communicating across many ECU boundaries.

Critically, SDVs are enabled by **embedded operating systems** that provide a unified software environment on these central controllers. Instead of each ECU running bare-metal code, the vehicle can run a general-purpose **embedded Linux** or specialized real-time OS across its compute platforms. Many in the industry are converging on Linux-based solutions for non-safety domains, owing to the rich ecosystem and hardware support. For example, the **Eclipse Leda** project (under the Eclipse Foundation’s SDV working group) provides a reference **Linux distribution for automotive** built with the Yocto Project. Eclipse Leda’s Yocto-based Linux stack integrates essential SDV components (middleware, container runtime, update services) into a ready-to-use image, offering a baseline for developers and researchers to experiment with software-defined vehicle features. By using standardized embedded Linux platforms, automakers can more easily adopt **containerization** and modern software practices in vehicles.

Containerization has emerged as a key technology in SDV architectures. Containers (e.g. using Docker or OCI-compliant runtimes) package applications with their dependencies, providing a consistent environment across development, testing, and in-

vehicle deployment. This approach, commonplace in cloud computing, is now being adapted to vehicles to manage the growing software complexity. Containerization **decouples applications from the underlying vehicle hardware**, allowing the same software component to run on different car models or SoC platforms as long as a compatible runtime is available. It also enables **safe isolation** of services (important for mixing critical functions like braking control with non-critical apps on the same hardware). According to the eSync Alliance, which promotes OTA standards, containerized software gives automakers a “*consistent and reliable runtime environment*” that is hardware-agnostic, making it easier to deploy updates and new features across a fleet. Notably, containers facilitate **frequent OTA updates** – manufacturers can push new ADAS algorithms or infotainment apps remotely without worrying about each ECU’s specific configuration. By abstracting away hardware differences, a containerized ADAS module (for example, a **Traffic Sign Recognition (TSR)** service) can be updated for all supported vehicles in one go, significantly reducing maintenance effort.

However, using containers in cars also requires careful orchestration and attention to real-time performance. Automotive software must meet strict safety and timing requirements. Tools like **Kubernetes** or cloud-native orchestration have been modified for the edge to deploy and monitor vehicle containers. Moreover, automotive-grade container frameworks (e.g. **Eclipse Kanto**, an edge orchestrator, or Apex.AI’s middleware) often leverage underlying *real-time operating systems (RTOS)* or hypervisors for critical tasks, ensuring that container overhead doesn’t impede time-sensitive functions. This fusion of **cloud-native technology with automotive real-time needs** is exactly the focus of emerging industry efforts like SOAFEE (discussed later). In short, the SDV architecture borrows the best of IT software practices – containerization, modularity, continuous integration – and adapts them to the car, resulting in a far more flexible platform than traditional vehicles.

2.3 Over-the-Air Updates and Modular Design

One of the defining advantages of a software-defined vehicle is the ability to update and enhance the car *after it has been sold*, through over-the-air software updates. **OTA updates** in an SDV go beyond just map or infotainment updates – they can include performance improvements, new driver assistance features, UI enhancements, and bug fixes to critical systems. This capability fundamentally changes the automotive ownership experience. For drivers, it means the vehicle can get **better and safer over time**, much like a smartphone that receives periodic upgrades. Automakers can fix software defects without recalling vehicles, deploy enhancements on the fly, and even unlock revenue streams by offering *features on demand*. For example, a manufacturer

might enable a new **parking assist feature** or improved battery management algorithm via software download, improving the car's functionality years after production. Valeo notes that an SDV's performance "*will improve over time and you will be able to customize features... to match your needs*", as updates continuously refine the vehicle. This is a stark contrast to legacy vehicles, which typically remain static in capability after leaving the factory.

Achieving safe and reliable OTA updates requires a **modular software design** in the vehicle. The system must be able to update one module (say, the blind-spot detection algorithm) without disturbing other functions. Hence SDV platforms emphasize **clearly separated software layers and services**, often using a *microservices* or *service-oriented architecture*. Each functional domain (ADAS, powertrain, body control, etc.) can have its software segmented into independent services with well-defined interfaces. Modern automotive middleware (like **AUTOSAR Adaptive** or open-source equivalents) provide the framework for this: applications are deployed as services, and communication is handled via standardized APIs. This not only eases updates but also allows **third-party development**. For instance, GM's *Ultifi* platform (an SDV software platform) exposes APIs that let authorized developers create new in-vehicle apps without touching the underlying hardware-specific code. Under the hood, Ultifi is built as an **Linux-based software platform** separated from the vehicle hardware. This separation of application-layer software from the hardware abstraction is what enables GM to deliver updates "frequently and seamlessly" OTA, since new software packages can be deployed on a standardized Linux OS across all vehicles. In effect, Ultifi and similar SDV platforms treat the car like another node on the internet – capable of receiving continuous improvements, much as our computers and phones do.

A modular SDV design also improves **safety and security** maintenance. Critical fixes (e.g. patching a security vulnerability in the telematics unit) can be delivered quickly. With increasing connectivity, vehicles must be guarded against cyber threats, and OTA provides a mechanism to respond to threats rapidly. Many regulatory bodies now **require** manufacturers to demonstrate software update mechanisms and cybersecurity measures (e.g., UN Regulation No. 155 mandates cyber management for vehicle software including updates). Thus, OTA capability is not just a nice-to-have but is becoming an industry standard for compliance. SDVs, by design, fulfill this through built-in update clients, secure bootloaders, and version management systems that allow safe downloading and installation of new firmware or application code. In summary, the combination of **modular software architecture** and **OTA updatability** is a cornerstone of the SDV concept, enabling a vehicle that can continuously evolve, adapt, and improve throughout its lifecycle.

2.4 Integration of ADAS Functions in SDV Frameworks

Advanced Driver-Assistance Systems (ADAS) – such as traffic sign recognition (TSR), automatic emergency braking (AEB), and blind spot detection (BSD) – are increasingly software-heavy, AI-driven functions. In legacy vehicles, ADAS capabilities were often delivered via discrete hardware modules or add-on ECUs (for example, a radar unit with its own processor for adaptive cruise control). In a software-defined vehicle, ADAS is treated as part of the overall software ecosystem: **ADAS algorithms run as applications or services on the central computing platform**, leveraging shared sensors and common compute resources. This integration yields several benefits:

- **Shared Sensor Data and Perception:** SDVs typically have a centralized data bus or middleware that collects inputs from all sensors (cameras, LiDAR, radar, ultrasonic, etc.) and makes them available to multiple functions. This means features like BSD and AEB can access the *same fused sensor view* of the environment. A centralized “environment model” can be computed and shared, improving consistency between ADAS features. For example, if a pedestrian is detected by the vision system, that information can be simultaneously used by the AEB function to trigger braking and by the warning system to alert the driver. In older architectures, each ADAS feature might operate on its own limited sensor input.
- **High-Performance Computing for ADAS:** Many ADAS tasks involve intensive computations (computer vision, neural network inference, sensor fusion). A software-defined approach centralizes these into an automotive **HPC (High-Performance Computer)** or domain controller that has ample processing power (multi-core CPUs, GPUs, NPUs) specifically to handle ADAS and autonomous driving algorithms. Rather than relying on low-power microcontrollers in each ECU, the central ADAS computer can run complex AI models in real-time. This is crucial for achieving higher levels of autonomy. The trade-off is that it demands a robust thermal and power design in the vehicle, but industry trends (and the advent of electric vehicles with high-voltage power) make this feasible. *Sensor fusion* approaches are evolving accordingly: current research pushes for **low-level sensor fusion** (combining raw data from sensors) to occur in real-time on central units, which improves accuracy and reduces the needed number of sensors by maximizing information use.
- **Software Updates and Improvements to ADAS:** As mentioned, one huge advantage is the ability to update ADAS features via software. If a new vision algorithm improves pedestrian detection, a software-defined vehicle can deploy this improvement to all cars in the field, potentially saving lives without any hardware change. Automakers have leveraged this; for instance, Tesla famously

improved its Autopilot capabilities through software updates. Traditional cars could not retrofit such enhancements easily. The SDV approach also allows **feature modularity** – owners might opt to purchase or activate certain ADAS features after vehicle purchase (e.g. enabling a *lane-keeping assist* that was pre-installed but software-disabled). This flexibility is only possible when the vehicle is essentially pre-loaded with the necessary sensors and the rest is just software configuration.

Integrating ADAS into an SDV framework does pose challenges. ADAS functions are **safety-critical**, so the software processes managing them must meet real-time deadlines and be isolated from faults. This is addressed through *mixed-criticality architecture*: for example, an SDV's central computer might run a safety-certified RTOS for functions like AEB and steering control, in parallel with a Linux environment for less critical tasks, communicating via a safe IPC mechanism. Industry standards like **AUTOSAR Adaptive** help in this regard by providing a platform where safety-critical applications can coexist with others and communicate over a controlled interface. Additionally, car makers are exploring *domain controller redundancy* (multiple parallel computers) to ensure ADAS resilience – e.g., one handling perception, another making decisions, providing fail-operational capability for higher automation levels. Still, the net effect is clear: **SDV design blurs the line between ADAS and the rest of the vehicle software**. Rather than an add-on, ADAS is an integral part of the vehicle's software suite, designed with the same principles of modularity and updatability. As a concrete example, Valeo's ADAS domain controller for the BMW Neue Klasse will host *parking assist*, *blind-spot monitoring*, *traffic sign recognition*, and more on one platform, illustrating how a single software platform now covers what used to be multiple electronic subsystems.

2.5 In-Vehicle Communication and Real-Time Messaging

Transitioning to software-centric vehicles requires a rethinking of in-vehicle communication. Legacy vehicles predominantly use **CAN buses** (Controller Area Network) and similar protocols (LIN, FlexRay) for ECU communication. These are efficient for short control messages but have limited bandwidth (CAN, for instance, maxes out at 1 Mbps in classical form) and are not suited for high-volume data or complex service discovery. SDVs, with their central compute and myriad software services, instead embrace an **Ethernet-based network architecture** for the car. Automotive Ethernet (often with Time-Sensitive Networking extensions) provides the high throughput and deterministic timing needed to shuttle data between sensors, central computers, and actuators in real time. It also supports *internet protocols*, which is crucial for adopting modern middleware.

On top of this physical network, **service-oriented communication** patterns are used. Rather than fixed point-to-point signals, SDVs deploy middleware that allows dynamic message exchange, publish/subscribe data distribution, and service calls. **SOME/IP** (Scalable service-Oriented Middleware over IP) is one such protocol adopted in many production cars – it enables services to find each other and communicate over Ethernet using a standardized format. **Data Distribution Service (DDS)**, a middleware standard known from aerospace and robotics, is another technology entering automotive for its real-time, peer-to-peer publish/subscribe capabilities. DDS is designed for high throughput and low latency, making it suitable for distributing sensor data or control commands without a broker. In fact, DDS is being integrated into the AUTOSAR Adaptive standard as an alternative to SOME/IP for service communication, particularly for ADAS and autonomy use cases that demand real-time performance. An interview with the CTO of RTI (a leading DDS vendor) highlighted that DDS can leverage features like multicast and shared memory to meet strict latency bounds, and it can run over Time-Sensitive Networking to guarantee bounded delays, which is *“crucial for real-time control and high-bandwidth data flows (e.g., video or LiDAR) in a shared vehicle network”*.

In parallel, for cloud connectivity and non-critical data exchange, many SDV implementations use protocols like **MQTT** – a lightweight publish/subscribe messaging protocol common in IoT. MQTT fits well when vehicles need to send telemetry to cloud servers or receive remote commands (for instance, a smartphone app unlocking a car or sending navigation destinations). It’s not designed for in-vehicle real-time control (since it typically relies on a broker and doesn’t ensure strict delivery times), but it is excellent for decoupling senders and receivers and working in bandwidth-constrained environments. The Eclipse Velocitas SDV SDK, for example, uses MQTT under the hood for its vehicle app communications: developers can publish and subscribe to vehicle data topics without worrying about low-level details. This allows an **asynchronous event-driven model** in the vehicle software – e.g., a diagnostic service publishes a “tire pressure low” event which any interested component (dashboard, cloud alert service, etc.) can subscribe to and handle appropriately. MQTT brokers can live in the vehicle or at the edge of the cloud; in some designs, the vehicle has an internal broker for in-car messaging and another broker connection for cloud uplink.

To manage both real-time and non-real-time communication, **automotive middleware** often implements a layered approach:

- Critical, real-time data (like control loops for braking or steering) use deterministic protocols (CAN for very low-level or TSN Ethernet with a real-time pub/sub like DDS). These ensure that, say, a brake command is delivered *within a few milliseconds* reliably.

- Less critical data and infotainment use IP-based services (TCP/UDP, HTTP/REST, gRPC, etc.) as needed, which provides flexibility and use of standard tools.
- A publish/subscribe *data bus* allows all software components to share information. This could be abstract: for example, a “vehicle data broker” service can present an API to apps for reading sensor values or issuing commands, internally translating them to the proper network messages. The Eclipse SDV ecosystem has a **Vehicle DataBroker** concept (via Eclipse Kuksa) that does exactly this, exposing vehicle signals in a unified way and supporting pub/sub updates.

In summary, robust communication in SDVs is achieved by **combining IT networking technologies with automotive-specific enhancements**. The car becomes a rolling data center: Ethernet links various “computers” (sensors, actuators, ECUs), and a mix of messaging protocols ensure data flows where it needs to. As SDVs advance, even more connectivity is expected (V2X communication with infrastructure, cloud-based services for heavy computing tasks, etc.), so the in-car network is truly the vehicle’s nervous system. It must handle real-time reflexes and high-level information exchange simultaneously – a challenge that modern automotive network design is actively addressing through service-oriented architectures and intelligent middleware.

In summary, the SDV landscape is being shaped by a combination of **academic research, industry standards, and open-source collaboration**. Academic and industrial research provides proofs-of-concept for new SDV technologies (e.g., container management in ECUs, scheduling algorithms for mixed-criticality containers, security frameworks for OTA), while standards bodies like AUTOSAR ensure these ideas can be deployed consistently at scale. Open-source projects (Eclipse SDV, AGL, etc.) then provide reference implementations that reduce development cost and risk for automakers. Thanks to these collective efforts, the concept of the software-defined vehicle is rapidly transitioning from vision to reality. Major manufacturers are already delivering cars that exemplify many SDV attributes – **centralized compute, software-upgradeable features, rich connectivity – and this chapter has surveyed the foundation that enables that transformation**. The next chapters will delve deeper into specific technologies and implementations, but the related works discussed here establish the crucial context and state-of-the-art for understanding Software-Defined Vehicles and their role in the future of automotive engineering.

CHAPTER 3: Analysis, Tools, and Design

3.1 Analysis

3.1.1 Problem Space

The automotive industry is witnessing a shift from hardware-centric design to software-defined paradigms. Traditional vehicles rely on rigid architectures with distributed Electronic Control Units (ECUs), leading to integration complexity and limitations in scalability. This project addresses the need for a modular, software-centric prototype of a Software-Defined Vehicle (SDV) that accurately models real-world automotive systems, integrating multiple Advanced Driver Assistance Systems (ADAS) in a scalable architecture.

3.1.2 Goals and Objectives

- Develop a scalable, modular prototype of an SDV
- Integrate three ADAS features: Blind Spot Detection (BSD), Emergency Braking System (EBS), and Traffic Sign Recognition (TSR)
- Support Over-the-Air (OTA) updates for containerized applications
- Ensure real-time data acquisition and control via perception and actuation layers

3.2 Tools and Platforms

3.2.1 Hardware Platforms

- **Raspberry Pi 4:** Acts as the central computing unit, handling AI inference, decision logic, and GUI.
- **STM32F401RC Microcontroller:** Dedicated to real-time sensor interfacing and motor actuation. Implements FreeRTOS for deterministic task scheduling
- **Ultrasonic Sensors (HC-SR04):** Measure distances for BSD and EBS.
- **Camera Module:** Provides visual input for TSR with YOLOv8 Nano.

3.2.2 Software Platforms

- **Eclipse Leda:** Yocto-based embedded Linux tailored for SDVs. Supports container deployment, OTA, and edge-ready execution
- **FreeRTOS:** Manages concurrent tasks on STM32 for precise sensor timing and motor control.
- **MQTT:** Lightweight protocol used for inter-module communication (sensor updates, alerts, control messages)

- **Qt (C++):** For infotainment GUI, handling user interactions and ADAS feature toggle

3.2.3 Development Tools

- **YOLOv8 + ONNX:** AI model trained for TSR; converted for fast inference on Raspberry Pi
- **Docker / Kanto:** Used for containerization and lifecycle management of individual ADAS modules.
- **spdlog:** Logging library for runtime debugging and system diagnostics.

3.3 System Design

3.3.1 Architectural Overview

The system is built using a **layered modular design**, dividing functionality across:

1. **Perception Layer:** STM32 + ultrasonic sensors for environmental data.
2. **Actuation Layer:** STM32 controls motors and actuators.
3. **Decision Layer:** Raspberry Pi handles AI, decision logic, and coordination.
4. **Infotainment Layer:** Qt GUI displays alerts and enables user interaction.

Modules communicate over MQTT, ensuring decoupled yet synchronized behavior. This is illustrated in the system's flow diagrams and class/communication models across BSD, EBS, and TSR modules

3.3.2 ADAS Module Design

Blind Spot Detection (BSD)

- Subscribes to sensors/left, sensors/right
- Publishes to info/bsd/left, info/bsd/right
- Threshold-based classification: Safe / Warning / Danger

Emergency Braking System (EBS)

- Subscribes to sensors/front
- Publishes to info/braking
- Also triggers actuator commands when critical distance is breached

Traffic Sign Recognition (TSR)

- Performs real-time inference using YOLOv8 Nano (via ONNX)

- Publishes recognized signs with confidence and timestamp to info/tsr

3.3.3 Containerization Strategy

Each ADAS feature is encapsulated in a container to ensure modularity and isolation:

- Communication via MQTT
- Managed using Eclipse Kanto
- OTA updates replace individual containers without system reboots

3.3.4 Infotainment Interface

- Qt-based GUI with real-time status updates
- Allows enabling/disabling of BSD, EBS, TSR via control topics (e.g., control/bsd)
- Receives alerts from info/* topics and visualizes them

3.4 Security & Maintenance

- **Namespace isolation:** Each container runs with minimal privileges
- **Hardware access policies:** Defined via container manifest (e.g., SPI for EBS, camera for TSR)
- **OTA security:** Eclipse Leda provides signing and rollback support
- **Future enhancements:** Cloud logging, AI-based failure recovery, dynamic threshold adaptation

3.5 Summary

The design of this SDV system reflects modern automotive architecture practices:

- It separates concerns between sensing, decision, actuation, and HMI.
- It leverages containerization and OTA for scalability and maintainability.
- It uses open standards and lightweight protocols for robust integration. This approach provides a strong foundation for future additions like lane-keeping, V2X communication, or full autonomy using the same modular pattern.

CHAPTER 4: Implementation

4.1 ACTUATION AND PERCEPTION LAYERS

4.1.1 Introduction

In the context of Software Defined Vehicles (SDVs), the separation of concerns between data acquisition, decision-making, and physical execution is critical to enabling safe and modular vehicle functionality. This chapter presents the Perception and Actuation layers of our SDV prototype, both implemented on the STM32F401RC microcontroller. These layers handle the real-time sensing of the vehicle's surrounding environment and the execution of low-level physical actions in response to commands from the decision-making unit.

The Perception Layer is responsible for monitoring the environment using ultrasonic sensors. In our system, three HC-SR04 ultrasonic sensors are employed—two mounted on the sides to implement Blind Spot Detection (BSD) and one mounted at the front to support the Emergency Braking System (EBS). STM32 generates trigger pulses, captures echo durations using hardware timers in input capture mode, and computes distances with high temporal precision. These measurements represent the vehicle's awareness of nearby objects.

The Actuation Layer, on the other hand, is responsible for carrying out physical responses based on decisions made by the higher-level processing unit (Raspberry Pi 4). These responses include activating a braking mechanism, illuminating a warning LED, or triggering a buzzer. Commands from the Raspberry Pi are received over the SPI interface, parsed by the STM32, and used to control GPIO outputs or connected actuators accordingly.

Together, these layers enable the vehicle to react to dynamic traffic conditions in real time. By offloading sensor handling and actuation to a dedicated embedded controller, the system achieves a modular and time-deterministic architecture, which is a key characteristic of modern SDV and ADAS design.

4.1.2 System Architecture Overview

The actuation and perception layers of our Software Defined Vehicle (SDV) prototype are implemented on the STM32F401RC microcontroller. These layers are responsible for capturing real-time environmental data from ultrasonic sensors and executing actuation commands received from the decision-making layer running on the Raspberry Pi 4.

To manage concurrency and timing, the STM32 firmware is structured using FreeRTOS, which ensures deterministic scheduling of sensor reading, SPI communication, and actuator actuation tasks. The system is built to operate in real-time, guaranteeing safe and responsive behavior of ADAS features.

This modular approach not only reflects real-world SDV architecture but also enables easy testing, debugging, and future expansion with additional sensors or actuation strategies.

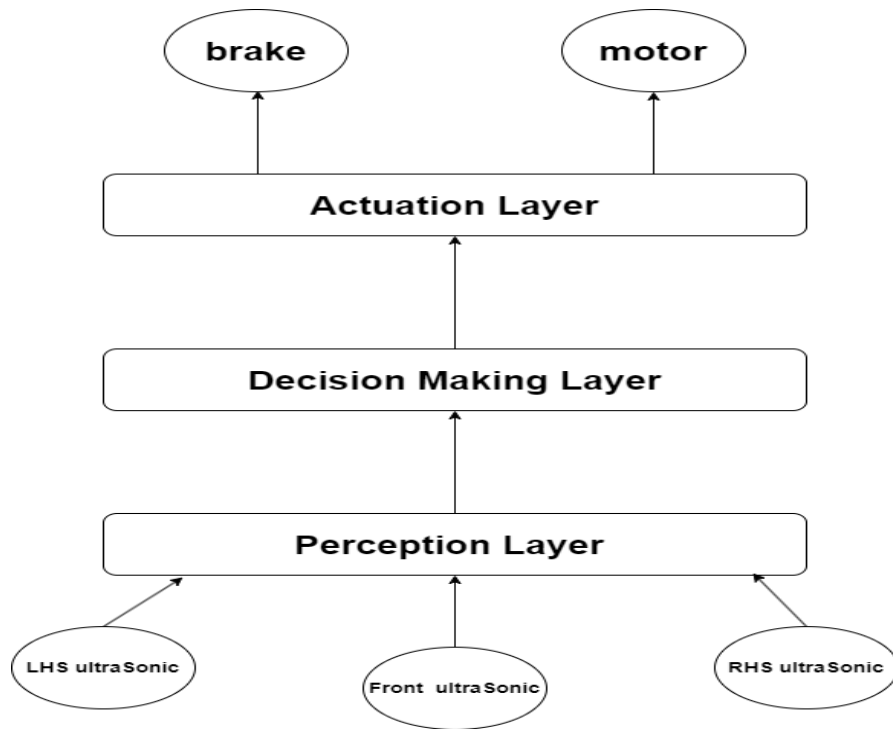


Figure 4.1.1: Perception and Actuation Layers Architecture

4.1.3 Perception Layer

The Perception Layer of our Software Defined Vehicle (SDV) is responsible for sensing the external environment and forwarding distance data to the decision-making system (Raspberry Pi). This is achieved through a tightly integrated setup of ultrasonic sensors, timers, interrupts, and RTOS-based task scheduling on the STM32F401RC microcontroller. This layer processes raw echo signals into precise distance measurements in real time.

I. Sensor Configuration and Hardware Mapping

Three HC-SR04 ultrasonic sensors are used to detect nearby obstacles:

- Front Sensor (EBS) – Detects frontal obstacles for emergency braking.
- Left Sensor (BSD) – Detects vehicles in the left blind spot.
- Right Sensor (BSD) – Detects vehicles in the right blind spot.

Each sensor uses:

- A trigger pin for sending a pulse.

- An echo pin connected to a timer's input capture channel for receiving echoes.

Timers used:

- These timers are configured in input capture mode using HAL and CMSIS APIs.

II. Signal Capture and Distance Calculation

The sensor readings follow this process:

- The trigger Task toggles each sensor's trigger pin to send a 10 microseconds pulse.
- Each sensor's echo pin is connected to a hardware timer in input capture mode. The rising edge marks the start of the echo; the falling edge marks the end.
- An interrupt service routine (ISR) handles these edges:
 - On the rising edge, the first capture value is stored.
 - On the falling edge, the second capture value is stored.
- The difference between the two values is used to calculate distance as follows:

$$Distance (cm) = \frac{time\ difference \times speed\ sound}{2}$$

This logic is encapsulated in the input capture handler function.

III. RTOS-Based Task Scheduling

Two FreeRTOS tasks manage the Perception Layer:

- Trigger Task
 - Triggers each ultrasonic sensor sequentially.
 - Waits 40 milliseconds between pulses to avoid signal interference.
 - Uses microsecond-level delays to comply with the HC-SR04 timing requirements.
- Data Sending Task
 - Continuously polls a FreeRTOS queue that receives processed distance readings from the ISR.
 - Acquires a mutex to safely send data over SPI.

IV. Data Queue and Synchronization

Distance readings are stored in a queue and contain sensor data as follows:

- sensorId: Identifies the sensor (e.g., 1 for front, 2 for left).
- distance: The computed distance in centimeters.

This queue decouples the timing between ISR (producer) and SPI sending task (consumer), increasing robustness and modularity.

V. Modular Initialization

All perception-related configurations are wrapped in two functions:

- First function Initializes GPIOs, timers, interrupts, and the queue.
- Second function Starts the FreeRTOS tasks for sensing and communication.

VI. Perception layer flowchart

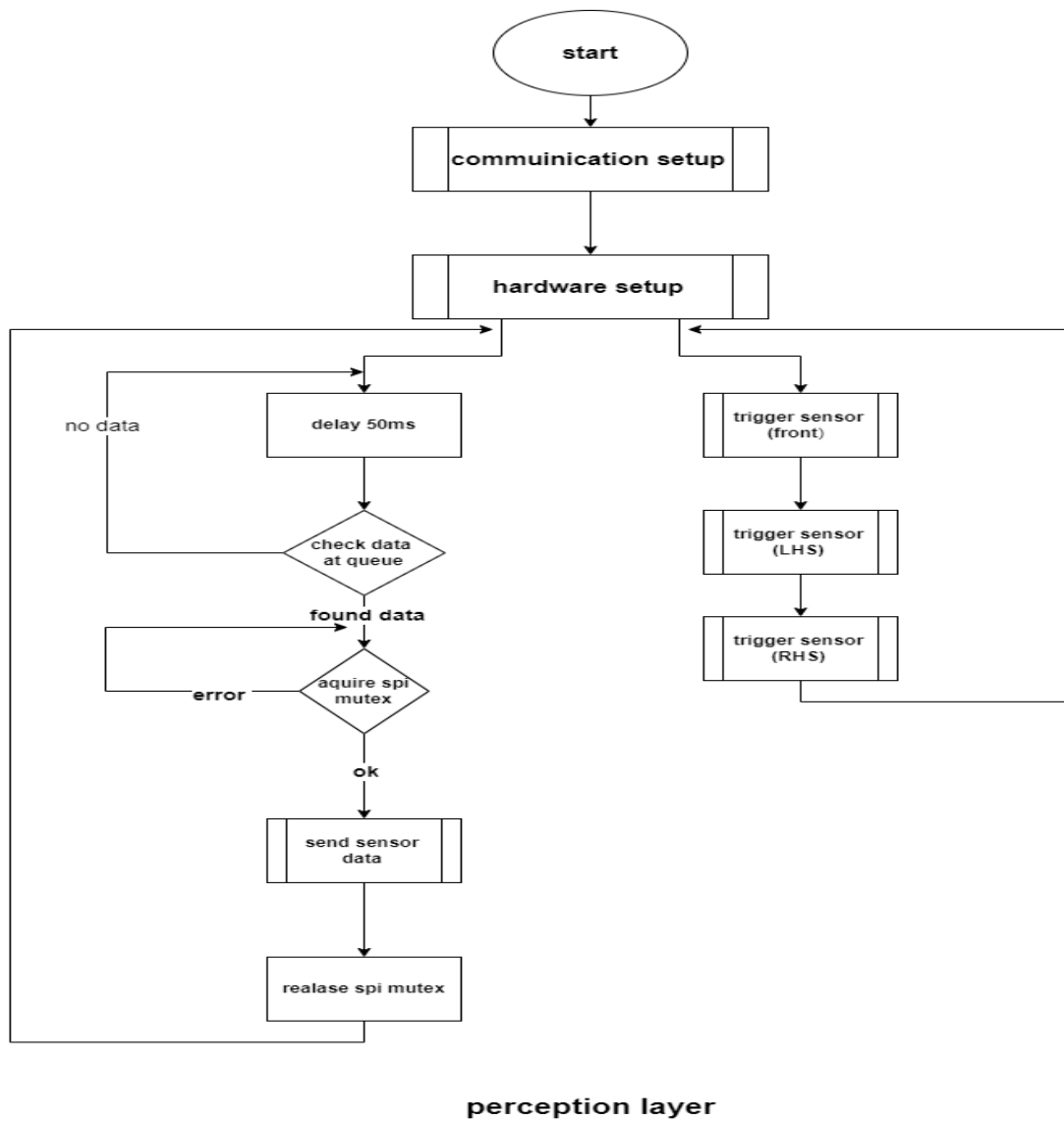


Figure 4.1.2: perception layer flowchart overview

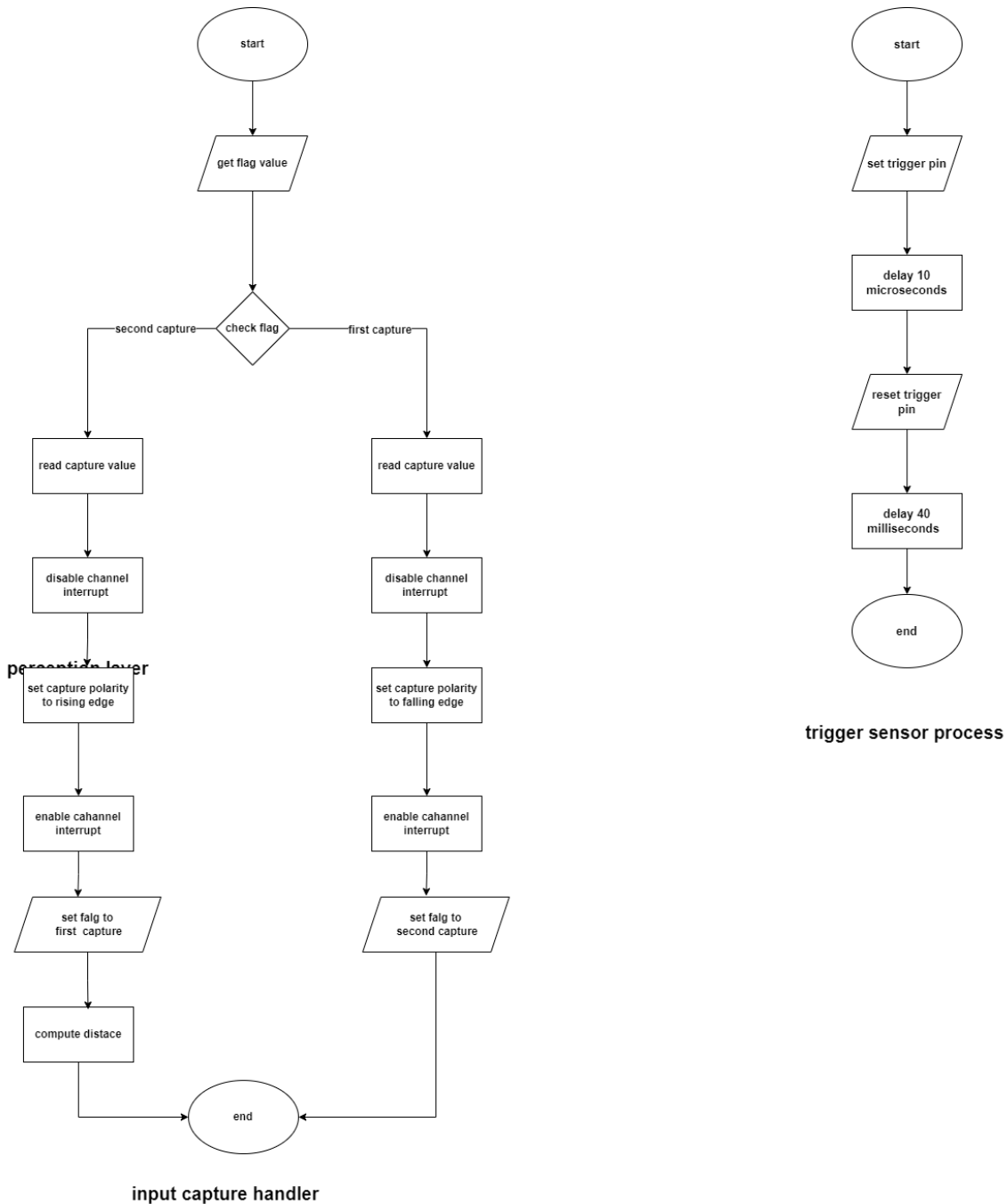


Figure 4.1.3: perception layer flowchart details

VII. Summary

The Perception Layer of this project:

- Enables real-time obstacle detection using efficient ultrasonic sensing.
- Uses CMSIS-HAL abstraction for portability and maintainability.
- Separates responsibilities via FreeRTOS to ensure precise timing, task scheduling, and data integrity.

- Acts as the foundation of the ADAS features Blind Spot Detection (BSD) and Emergency Braking System (EBS).

Together, this design allows the system to reliably sense its environment and forward that information to the decision-making layer for intelligent action.

4.1.4 Actuation Layer

The Actuation Layer in our Software Defined Vehicle (SDV) prototype is responsible for executing physical driving actions in response to high-level decisions received from the Raspberry Pi. These actions include movement commands like forward, reverse, turning, and stopping, all performed by two DC motors (left and right wheels) controlled through an STM32F401RC microcontroller.

This layer interprets commands received over the SPI interface, processes them using FreeRTOS, and then performs low-level GPIO control and PWM signal generation to drive the motors safely and accurately. All motor-related logic is encapsulated using CMSIS HAL APIs for maintainability and clarity.

I. Motor Driver Architecture

Each motor (left and right) is driven by:

- Two GPIO pins for direction control (IN1, IN2).
- One PWM output pin for speed control via a hardware timer (EN).

The configuration is performed through two motor structures, each containing:

- GPIO ports and pins for IN1, IN2, and EN.
- Timer instance and PWM channel for speed modulation.
- HAL handles initialization and runtime control.

II. Initialization Process

The initialization process is as follows:

- Configures IN1 and IN2 as push-pull output pins.
- Configures the EN pin as an alternate function (PWM).
- Initializes the timer for PWM mode.
- Sets default speed via macro and prepares the timer for runtime activation.

The PWM signal controls motor speed, while IN1 and IN2 determine direction:

- IN1 = 1, IN2 = 0 → Clockwise (CW).
- IN1 = 0, IN2 = 1 → Counter-clockwise (CCW).

The system halts PWM and resets direction pins before switching states to prevent electrical damage or mechanical jitter.

III. Command Execution and State Machine

The execute command task is responsible for executing commands received from the Raspberry Pi via SPI and performing the following:

- Receives commands using SPI.
- Evaluates command bytes and updates a state variable to ensure safe transitions.
- Controls both motors simultaneously to:
 - START: Start motors at initial speed and direction.
 - FORWARD / REVERSE: Change direction.
 - TURNLEFT / TURNRIGHT: Apply opposite directions to left and right motors.
 - STOP: Halts both motors and resets state.

Each command triggers GPIO changes and PWM updates using HAL APIs, ensuring clean and reliable execution.

IV. RTOS-Based Task Scheduling

Only one FreeRTOS task manages the actuation Layer:

- Execute command Task
 - Acquire mutex of SPI.
 - Receive command from raspberry PI 4.
 - Check command value and execute it.

V. actuation layer flowchart

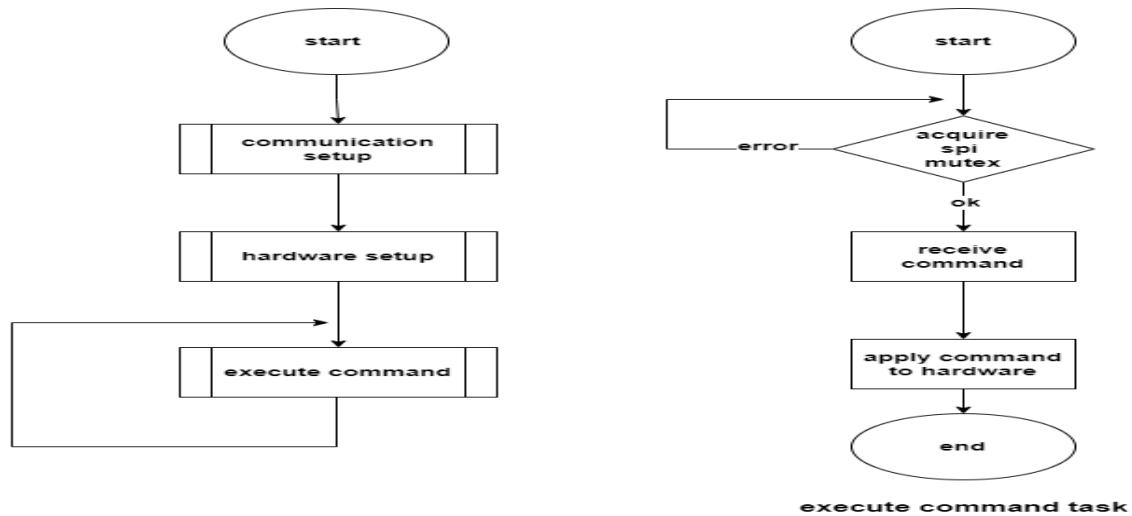


Figure 4.1.4 actuation layer flowchart

VI. Summary

The Actuation Layer:

- Directly controls two motors using PWM for speed and GPIOs for direction.
- It is tightly coupled with FreeRTOS for real-time execution.
- Reacts to structured SPI commands from the decision-making layer.
- Ensures safety by managing electrical transitions and command states.
- This layer effectively converts high-level vehicle decisions into physical motion, closing the loop from perception to action.

4.1.4 SPI Communication Layer

The SPI Communication Layer serves as the core communication interface between the STM32F401RC microcontroller and the Raspberry Pi 4, enabling synchronized data exchange between the perception/actuation layers and the central decision-making unit. Implemented using HAL SPI APIs and structured FreeRTOS mutex control, this layer guarantees data consistency, synchronization, and timing safety during bidirectional communication.

I. Architecture Overview

Data is formatted using custom start–data–end packet frames to ensure communication validity.

- STM32 (Slave): Acts as a passive endpoint, waiting for data requests or control commands from the master (Raspberry Pi).
- Raspberry Pi (Master): Initiates SPI transactions to either:
 - Retrieve sensor data from STM32 (e.g., ultrasonic distances).
 - Send actuation commands (e.g., movement direction, braking).

II. SPI Peripheral Initialization

SPI2 is configured in slave mode, full duplex, with:

- 8-bit data frame size.
- Clock polarity: low.
- Phase: 1st edge.
- Hardware NSS (slave select).
- MSB-first transmission.

III. Data Frame Structure

All SPI messages conform to a structured frame of 10 bytes:

- 1 byte start frame: 0xA0.
- 8 bytes payload:

- For sensor data: 4-byte sensor ID, 4-byte distance.
- For commands: up to 8 bytes of control data.
- 1 byte end frame: 0xB0.

This structure ensures:

- Validity verification on both ends.
- Consistent buffer sizes.
- Safe parsing and reconstruction of transmitted content.

IV. Sensor Data Transmission

Sensor readings collected by the perception layer are sent to the Raspberry Pi using send sensor data function that works as follows:

- A packet is prepared:
 - Start frame byte (0xA0).
 - 8-byte serialized data (sensor ID + float).
 - End frame byte (0xB0).
- Data is serialized using serialize sensor data function.
- The full frame is sent using HAL API.

V. Command Reception

Control commands from the Raspberry Pi are received using receive command function.

- It performs a full-duplex SPI transfer using HAL API.
- A predefined frame is transmitted while the STM32 simultaneously reads incoming commands into the command buffer.
- Commands are parsed in the actuation layer task and executed accordingly.

VI. Synchronization and Safety

- The SPI bus is shared between transmission and reception logic, protected using a FreeRTOS mutex.
- All SPI operations are blocking operations to guarantee message delivery during transaction windows.
- Command buffers are cleared after each cycle to avoid re-processing old data.

VII. Summary

The SPI communication layer:

- Enables bidirectional communication between STM32 (slave) and Raspberry Pi (master).
- Uses custom framed packets to ensure data structure and integrity.
- It is synchronized with FreeRTOS mutexes for safe concurrent access.
- Seamlessly integrates with both the Perception and Actuation layers to close the ADAS data loop.
- This layer is critical to ensure the timely and accurate flow of information in a software-defined vehicle system.

4.1.5 Conclusion

This chapter presented the embedded implementation of the Perception, Actuation, and SPI Communication layers that form the foundation of our Software Defined Vehicle (SDV) prototype's low-level operation. Built entirely on the STM32F401RC microcontroller using CMSIS HAL and FreeRTOS, these layers serve as the vehicle's sensory interface, physical control interface, and real-time communication bridge to the decision-making system running on the Raspberry Pi 4.

The Perception Layer was implemented using three HC-SR04 ultrasonic sensors, each interfaced through timer input capture to accurately measure distances. The measured data was queued using a real-time message buffer and transmitted efficiently to the upper-layer controller using a synchronized SPI protocol.

The Actuation Layer received structured commands from the Raspberry Pi over SPI, which were then decoded and translated into precise motor control actions using GPIO and PWM logic. Two motors were configured for bidirectional motion and speed control, allowing the system to perform various vehicle maneuvers such as forward, reverse, turning, and stopping.

The SPI Communication Layer ensured safe and deterministic full-duplex data exchange by defining a framed packet structure and applying mutex-protected transmission and reception logic. This robust communication architecture enabled modularity, maintainability, and real-time coordination between software-defined layers.

Together, these layers illustrate a realistic, modular implementation of perception-to-actuation flow in a modern embedded automotive system, showcasing the practicality of SDV and ADAS principles in a compact and educational prototype. The architecture also lays the groundwork for extending the system with additional features or layers, such as cloud connectivity, CAN bus integration, or advanced decision logic.

4.2 Communication Module - SPI-MQTT Gateway

4.2.1 Introduction

The communication module serves as a bridge between the STM32 microcontroller and the Raspberry Pi 4, enabling real-time, bidirectional data exchange. Designed to support Advanced Driver Assistance Systems (ADAS) and IoT edge applications, this module facilitates:

- **Sensor data acquisition** from the STM32 via the Serial Peripheral Interface (SPI) protocol.
- **Data publishing** to an MQTT broker for downstream processing.
- **Command reception** from the broker and forwarding to STM32 for actuator control.

To ensure robust and reliable operation, the gateway incorporates self-healing mechanisms, thread-safe communication, diagnostic logging, and containerized deployment for improved reproducibility and scalability.

I. Design Choice: MQTT as the Communication Protocol

The communication module uses the **MQTT (Message Queuing Telemetry Transport)** protocol as the backbone for all inter-module messaging within the Software-Defined Vehicle (SDV) system.

MQTT was chosen for the following reasons:

- **Lightweight and Efficient:** MQTT operates over TCP/IP with minimal packet overhead, making it ideal for resource-constrained environments such as embedded systems and edge devices.
- **Publish-Subscribe Model:** Its decoupled architecture allows each module (e.g., EBS, BSD, infotainment) to independently publish or subscribe to relevant topics without tight coupling. This promotes modularity and scalability.

- **Built-In Reliability Features:** MQTT supports QoS levels, persistent sessions, and Last Will and Testament (LWT) messages, which improve robustness in case of unexpected disconnections.
- **Wide Ecosystem Support:** MQTT is supported across a wide range of platforms and libraries, including Paho-MQTT for Python, which integrates seamlessly with the Raspberry Pi.
- **Futureproofing:** MQTT's popularity in IoT and automotive systems ensures long-term support and easier integration with cloud platforms or OTA update services in the future.

This made MQTT a natural choice for enabling real-time, event-driven communication between the STM32-based sensing layer and higher-level processing and display modules.

4.2.2 Module Architecture and Implementation

This section describes the internal structure of the **SPI-MQTT Communication Module** and its interaction with other system components.

I. Module Components & Functions

The SPI-MQTT Communication Module is composed of several internal components, each responsible for a distinct task. While implemented within a single source file, the design follows a modular and multi-threaded architecture that ensures responsive, safe, and non-blocking communication between the STM32 and other ADAS modules.

- **Configuration Manager** - Loads runtime settings (such as MQTT broker info, SPI bus/device configuration, and topic names) from config.json. If the configuration file is missing or malformed, the module falls back to safe default values to maintain system reliability.

- **SPI Handler**- Manages all low-level SPI communication with the STM32 microcontroller. Responsibilities include:
 - Opening and configuring the SPI interface
 - Sending actuator command bytes to the STM32
 - Receiving and parsing sensor packets
 - Ensuring **thread-safe access** to the SPI bus using `threading.Lock()` to prevent race conditions between multiple threads
- **MQTT Client** - Handles all MQTT-related tasks, including:
 - Connecting to the MQTT broker (e.g., Mosquitto)
 - Subscribing to control topics: actuators, info/braking
 - Publishing data to topics: sensors/front, sensors/left, sensors/right, and sensors/heartbeat
 - Automatically reconnecting in case of disconnection
 - **Runs in a background thread** via `client.loop_start()` to avoid blocking the main application logic
- **Actuator Command Processor** - Implements a **dedicated background thread** that continuously monitors a thread-safe Queue for actuator commands received from MQTT.
 - Commands are enqueued in the MQTT callback function (`on_message`)
 - The actuator thread dequeues commands and sends them to the STM32 via SPI
 - Uses the same `threading.Lock()` to ensure mutual exclusion during SPI writes
 - Handles both normal actuator commands (e.g., "start", "stop") and emergency stop triggers ("DANGER")
- **Sensor Data Publisher** - Executed in the **main loop thread**, this component performs:
 - Periodic polling of sensor data from the STM32
 - Comparison with previously published values
 - Publishing of updated sensor readings to the appropriate MQTT topics

- Ensures the system is not overloaded by using short, controlled delays (~50ms)
- **Diagnostics and Heartbeat**
 - Periodically publishes a heartbeat message on the sensors/heartbeat topic
 - Includes the module's operational status, timestamp, and last-known sensor readings
 - Indicates whether the system is "Alive" or "No Connection"
- **Shutdown Manager-** Ensures a graceful shutdown sequence when the system exits or fails. Responsibilities include:
 - Stopping the MQTT client loop
 - Closing the SPI interface
 - Logging the shutdown event cleanly

II. System Context and Module Integration

The SPI-MQTT communication module functions as a central bridge between the STM32 and the broader Software-Defined Vehicle (SDV) system. It integrates with the system in the following ways:

Sensor Data Acquisition

Ultrasonic sensors connected to the STM32 provide distance measurements. The STM32 sends this data via SPI to the Raspberry Pi. The communication module parses the received packets and publishes them to the following MQTT topics:

- **sensors/front**
- **sensors/left**
- **sensors/right**

Actuator Command Reception

The module subscribes to actuator-related topics, such as:

- **actuators:** for general movement commands (e.g., "start", "turnLeft", "stop")
- **info/braking:** used by the Emergency Braking System to trigger an immediate stop in case of danger

Received commands are enqueued and processed asynchronously by a dedicated background thread, ensuring real-time responsiveness without blocking other system operations.

Integration with ADAS Modules

Modules like Blind Spot Detection (BSD) and Emergency Braking System (EBS) rely on data published by the communication module. These modules process sensor values and may issue control commands.

Integration with Infotainment System

The infotainment system can send high-level user commands such as "start" or "stop" through MQTT topics. These commands are published to the actuators topic, which the communication module subscribes to. Upon reception, the commands are queued and forwarded to the STM32 to initiate or halt vehicle movement.

Optional Control Loop

The communication module may also forward critical commands (e.g., "DANGER") back to the STM32 to trigger emergency braking or automated response.

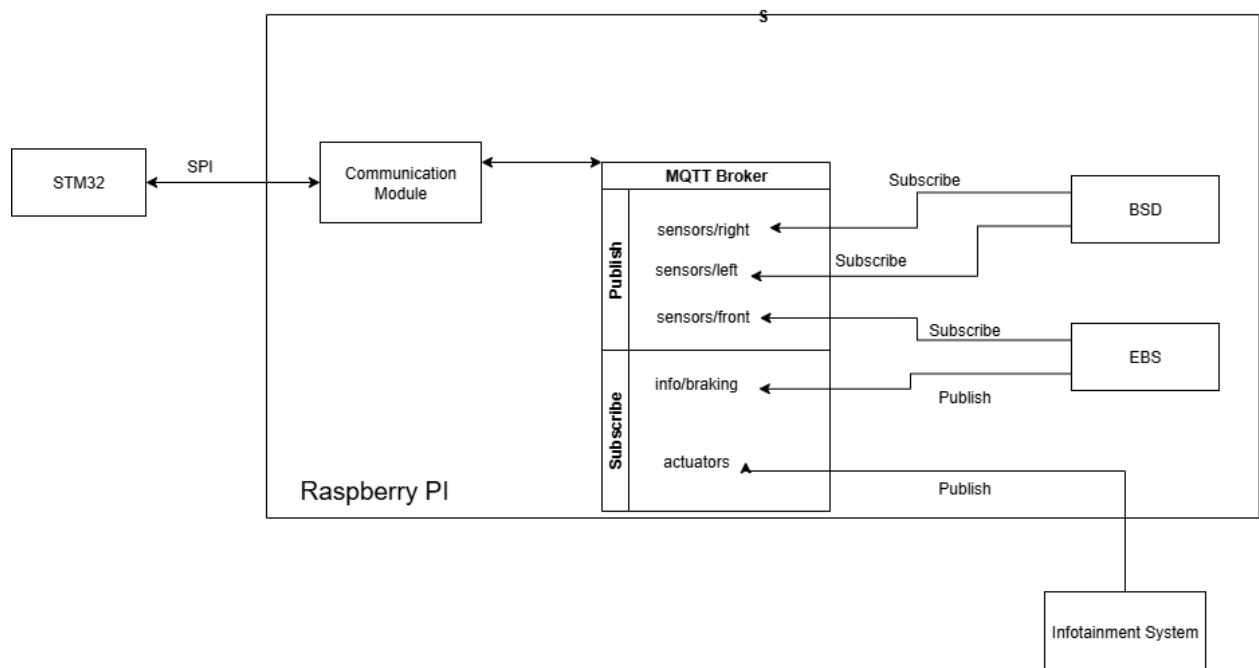


Figure 4.2.1: Block diagram of the Communication module within the Software-Defined vehicle architecture.

4.2.3 Summary

The SPI-MQTT Communication Module is a core component of the Software-Defined Vehicle (SDV) system, acting as a real-time data bridge between the STM32 microcontroller and high-level ADAS modules running on the Raspberry Pi. By leveraging the efficiency of the SPI protocol and the flexibility of MQTT, the module supports seamless bidirectional communication between sensors, actuators, and processing layers.

Key design features such as multi-threading, thread-safe SPI access, message queuing, and diagnostic heartbeats contribute to the module's responsiveness, robustness, and maintainability. The use of containerized deployment further enhances reproducibility across development and production environments.

Through its modular architecture and publish-subscribe messaging paradigm, the communication module enables flexible system integration with components such as Blind Spot Detection (BSD), Emergency Braking System (EBS), and the Infotainment system—supporting both safety-critical functionality and future scalability.

4.3 EMERGENCY BRAKING SYSTEM

4.3.1 Introduction

The Emergency Braking System (EBS) is a critical safety component within the vehicle's ADAS (Advanced Driver Assistance Systems). It continuously monitors the area in front of the vehicle using data from a front-mounted ultrasonic sensor. When an object is detected within a predefined safety threshold, the EBS module publishes a warning message via the MQTT communication layer. This message is displayed on the infotainment system, alerting the driver to take immediate action and helping to prevent potential collisions.

To ensure flexibility and driver control, the EBS module can be enabled or disabled through the infotainment interface. This allows the driver to activate the system when desired, making it a configurable safety feature rather than a mandatory one.

4.3.2 Module Architecture and Implementation

This section describes the internal structure of the Emergency Braking System (EBS) software module and its interaction with other system components.

I. Module Components

- Communication Module - Handles all MQTT related operations, including:
 - Connecting to MQTT broker
 - Subscribing to topics
 - Publishing to topics
- Emergency Braking Module - Acts as the core logic unit that processes incoming sensor data, compares it against configured thresholds, and generates status alerts accordingly:
 - Danger: if distance < critical threshold
 - Warning: if distance < warning threshold
 - Safe: otherwise
- Configuration Manager Module - Loads runtime settings (such as broker name, topic names, distance thresholds) from config.json and provides them to the EBS module.

The following UML class diagram illustrates the internal structure of the EBS module and its interactions.

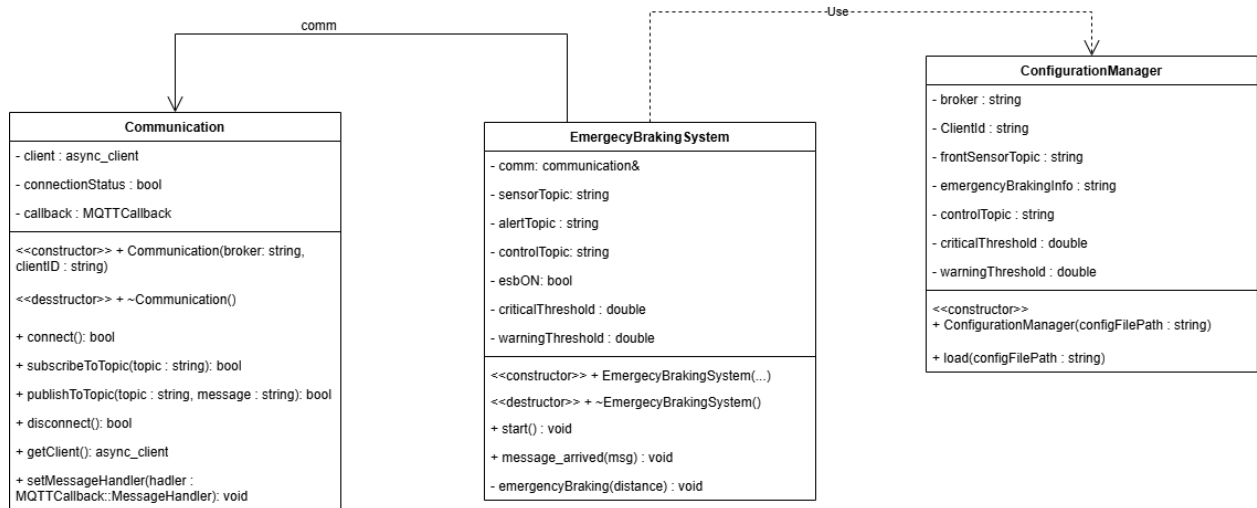


Figure 4.3.1: UML class diagram showing the internal structure of the Emergency Braking Software module.

II. System Context and Module Integration

The Emergency Braking System (EBS) module functions as part of the larger Software-Defined Vehicle (SDV) system. The EBS module interacts with the system as follows:

- **Sensor data reception:** The front ultrasonic sends distance data to the STM32 microcontroller. STM32 transmits this data via SPI to a communication app running on the Raspberry Pi. This data is then published by the communication app to the MQTT topic sensors/front. The EBS subscribes to the sensors/front topic to get the data.
- **Alert publishing:** When a potential collision is detected, EBS publishes a warning status to the info/braking topic.
- **Infotainment Integration:** The Infotainment system subscribes to the info/braking topic and displays visual alerts to the driver.
- **Optional Automatic Braking:** The communication app could also subscribe to info/braking topic to forward a command to the STM32 to trigger automatic braking

The block diagram below illustrates how the EBS module fits into the system, its external interfaces, and its internal components.

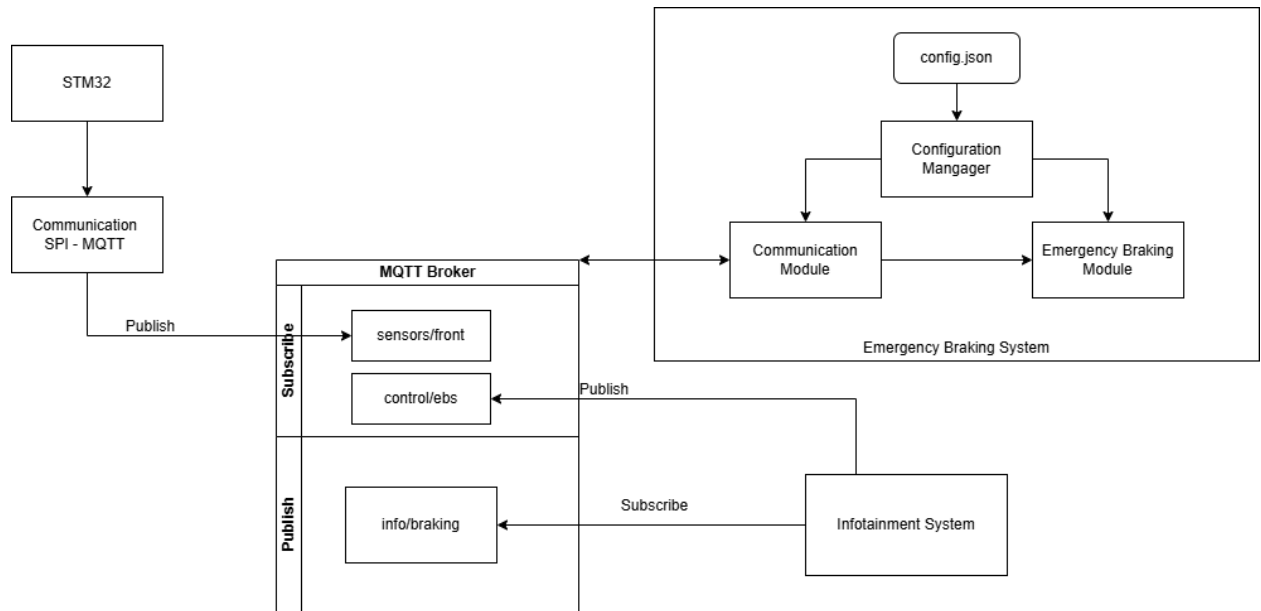


Figure 4.3.2: Block diagram of the Emergency Braking Module (EBS) module within the Software-Defined vehicle architecture.

4.3.3 Enable/Disable Mechanism

The Emergency Braking System (EBS) is a configurable (enable/disable) feature that allows the driver to control whether the system is active.

- The EBS module subscribes to the control/ebs MQTT topic.
- The infotainment system publishes ON or OFF messages to this topic based on the driver's selection.
- When the EBS module receives an OFF message, it disables its detection and alert logic.
- When it receives an ON message, it resumes monitoring sensor data and triggering alerts if necessary.

This mechanism ensures that the driver maintains control over the EBS feature and can choose to activate or deactivate it dynamically during operation.

4.3.4 Flowchart

The flowchart below illustrates the operational logic of the Emergency Braking System (EBS) module. It represents the decision-making process based on received distance values and system enable status. The module continuously evaluates incoming sensor data, checks whether the system is enabled, and determines whether to publish a warning message.

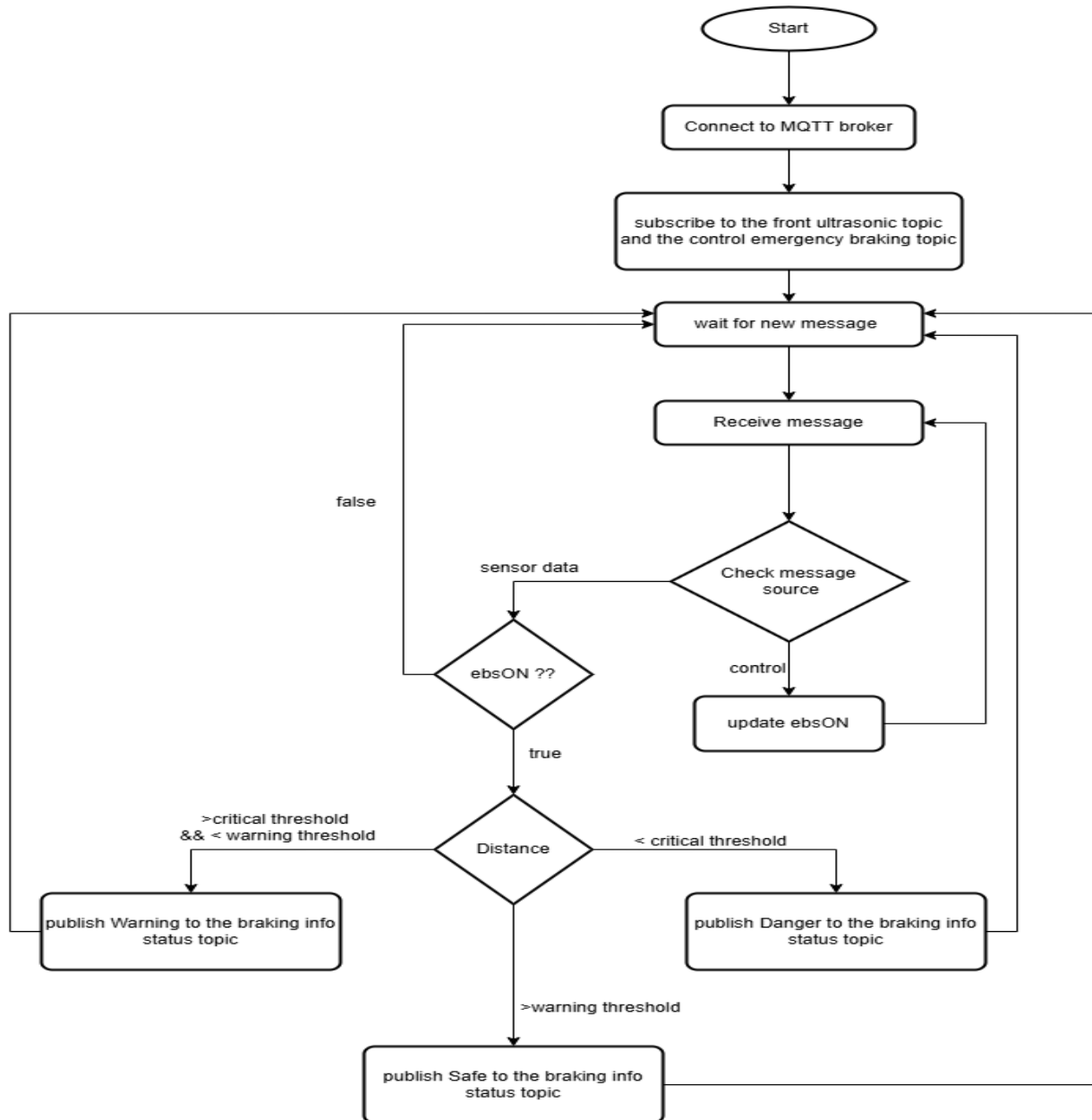


Figure 4.3.3: Flowchart representing the internal logic of the Emergency Braking System (EBS)

4.3.5 Conclusion and Future Work

The Emergency Braking System (EBS) module effectively aligns with the Software-Defined Vehicle (SDV) paradigm by emphasizing modularity, task isolation, and reusability. Each component—communication, configuration, and braking logic—is encapsulated and independently testable, allowing for seamless integration with other vehicle systems.

The use of a logging library (spdlog) during development has provided valuable runtime insight. In future iterations, this logging mechanism can be extended to write logs to persistent storage or upload them to a cloud service for real-time diagnostics, driver behavior analysis, or long-term performance monitoring.

Future enhancements may include:

- Integration with additional sensor types (e.g., camera or radar) for redundancy.
- Adaptive threshold tuning based on speed or environmental conditions.
- Implementation of actual braking actuation in coordination with the STM32 subsystem.

These improvements would further strengthen the EBS module's role in the overall ADAS framework, improving safety and system intelligence.

4.4 Blind Spot Detection

4.4.1 Introduction

As part of the Software-Defined Vehicle (SDV) with Advanced Driver Assistance Systems (ADAS) project, the Blind Spot Detection (BSD) module is responsible for enhancing driver awareness and road safety. It continuously monitors the vehicle's left and right blind spots using data from ultrasonic sensors. When an object is detected within a predefined distance threshold, the module publishes an alert message. These alerts get displayed to the driver on the infotainment system. These real-time alerts help inform the driver of potential hazards in adjacent lanes, reducing the risk of collisions during lane changes.

To ensure flexibility and driver control, the BSD module can be enabled or disabled through the infotainment interface. This allows the driver to activate the system when desired, making it a configurable safety feature rather than a mandatory one.

4.4.2 Module Architecture and Implementation

This section describes the internal structure of the Blind Spot Detection (BSD) software module and its interaction with other system components.

I. Module Components

- Configuration Manager Module- Loads runtime settings (such as broker name, topic names, distance thresholds) from config.json and provides them to the BSD module.
- Communication Module - Handles all MQTT related operations, including:
 - Connecting to MQTT broker
 - Subscribing to topics
 - Publishing to topics
- Blind Spot Detection Module - Acts as the core logic unit that processes incoming sensor data, compares it against configured thresholds, and generates status alerts accordingly:

- Danger: if distance < critical threshold
- Warning: if distance < warning threshold
- Safe: otherwise
- Left Blind Spot Thread - A dedicated thread running an instance of the BSD module for monitoring the left blind spot. It subscribes to the left sensor topic and publishes alerts to the corresponding status topic.
- Right Blind Spot Thread - A dedicated thread that runs a second instance of the BSD module to monitor the right blind spot. It operates independently, subscribing to the right sensor topic and publishing to its corresponding status topic.

The following UML class diagram illustrates the internal structure of the BSD module and its interactions.

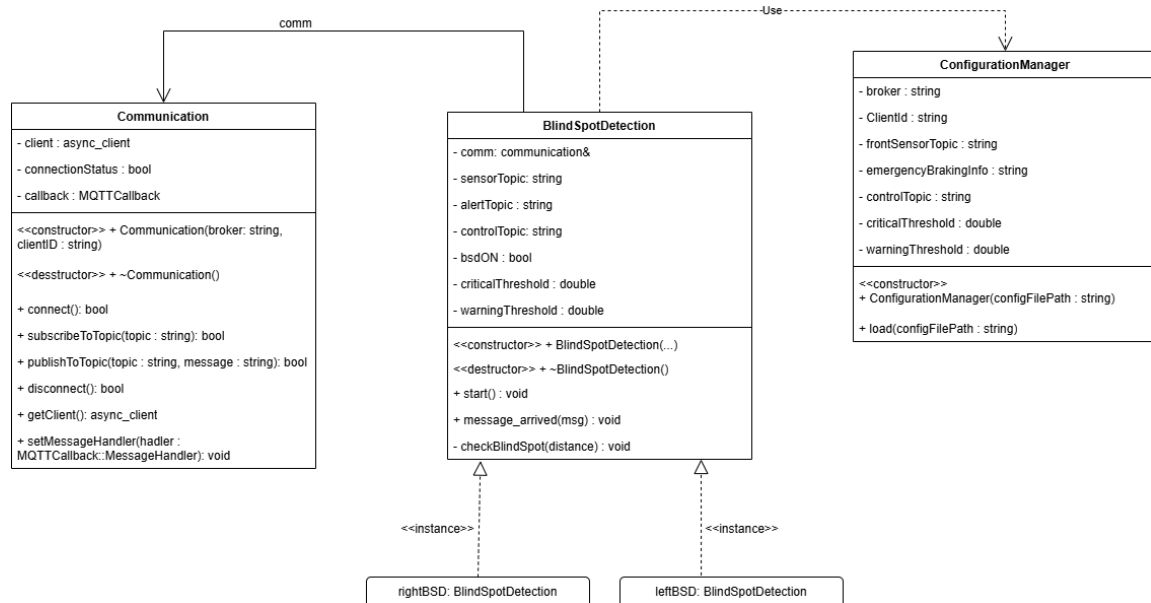


Figure 4.4.1: UML class diagram showing the internal structure of the Blind Spot Detection Software module.

II. System Context and Module Integration

The Blind Spot Detection (BSD) module functions as part of the larger Software-Defined Vehicle (SDV) system. The BSD module interacts with the system as follows:

- Sensor data reception:
 - The left and right ultrasonic sensors continuously measure distances to nearby objects.
 - This data is transmitted to the STM32 microcontroller, which aggregates and forwards it via SPI to the communication application running on the Raspberry Pi.
 - The communication app then publishes the sensor readings to the MQTT topics:
 - sensors/left
 - sensors/right
 - The BSD module subscribes to these topics to receive up-to-date distance information for processing
- Alert publishing:
 - Based on the configured warning and critical thresholds, the BSD evaluates incoming sensor data to determine the blind spot status.
 - When a potential collision is detected, the BSD publishes the appropriate alert to the following MQTT topics
 - info/bsd/left
 - info/bsd/right
- Infotainment Integration:

- The infotainment system subscribes to both info/bsd/left and info/bsd/right.
- Upon receiving an alert, it displays a visual warning to inform the driver of an object detected in the corresponding blind spot area.

The block diagram below illustrates how the BSD module fits into the system, its external interfaces, and its internal components.

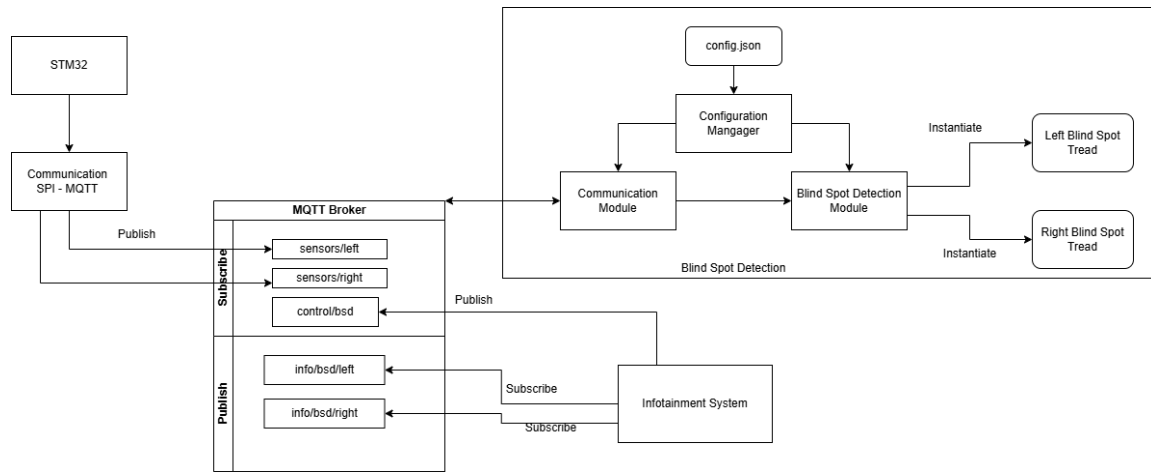


Figure 4.4.2: Block diagram of the Blind Spot Detection (BSD) module within the Software-Defined vehicle architecture.

4.4.3 Enable/Disable Mechanism

The Blind Spot Detection (BSD) is a configurable (enable/disable) feature that allows the driver to control whether the system is active.

- The BSD module subscribes to the control/bsd MQTT topic.
- The infotainment system publishes ON or OFF messages to this topic based on the driver's selection.
- When the BSD module receives an OFF message, it disables its detection and alert logic.
- When it receives an ON message, it resumes monitoring sensor data and triggering alerts if necessary.

This mechanism ensures that the driver maintains control over the BSD feature and can choose to activate or deactivate it dynamically during operation.

4.4.4 Flowchart

The flowchart below illustrates the operational logic of the Blind Spot Detection (BSD) module. It represents the decision-making process based on received distance values and system enable status. The module continuously evaluates incoming sensor data, checks whether the system is enabled, and determines whether to publish a warning message.

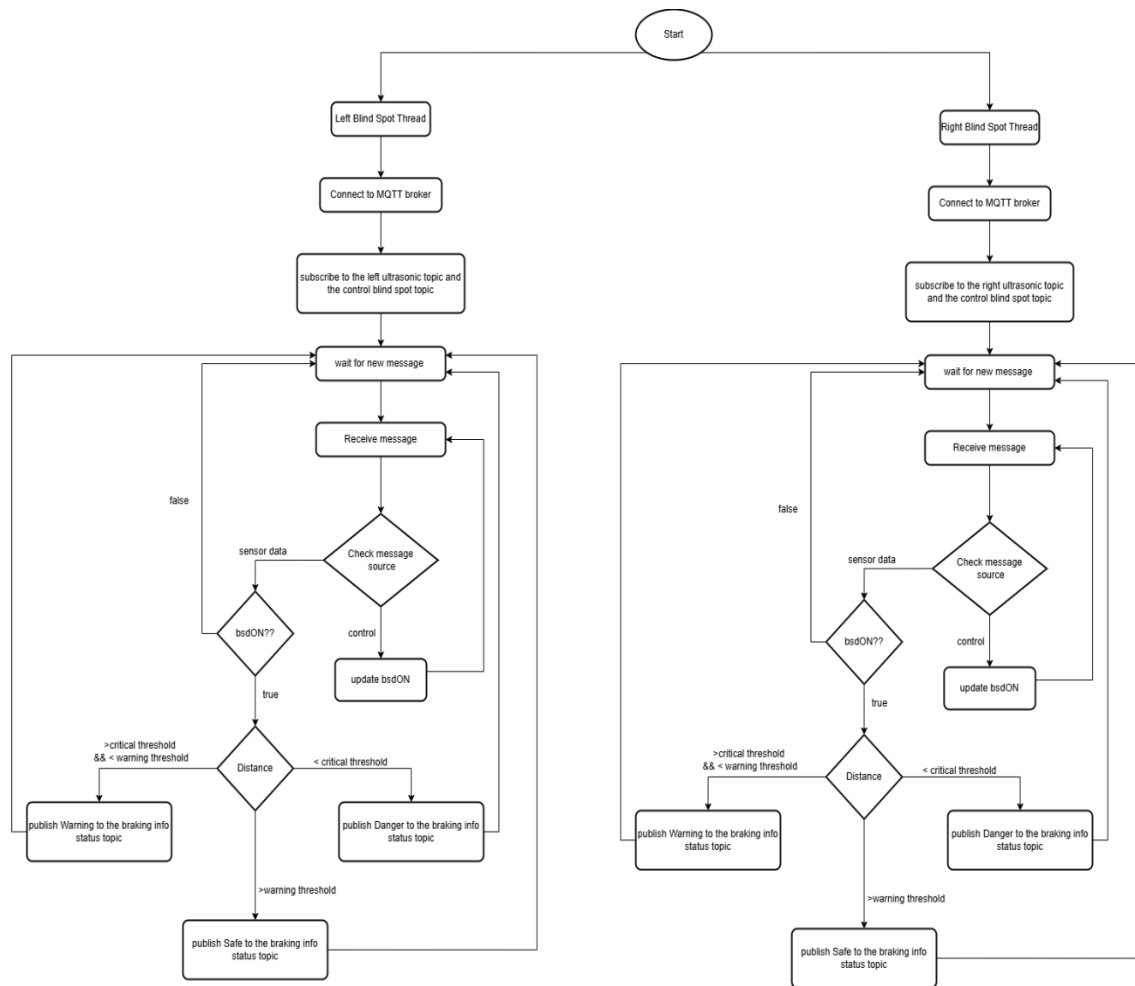


Figure 4.4.3: Flowchart representing the internal logic of the Blind Spot Detection (BSD)

4.4.5 Conclusion and Future Work

The Blind Spot Detection (BSD) module effectively aligns with the Software-Defined Vehicle (SDV) paradigm by promoting modularity, concurrent processing, and ease of integration. Each component—communication, configuration, and detection logic—is encapsulated and independently deployable, allowing seamless coordination with other vehicle subsystems within the ADAS framework.

The adoption of a structured logging mechanism using `spdlog` during development has facilitated real-time debugging and system observability. In future versions, logging can be extended to persist data locally or transmit it to cloud-based analytics platforms for driver assistance profiling, fault analysis, or long-term performance evaluation.

Future enhancements may include:

- Fusion with other sensor modalities (e.g., side-facing radar or vision) to improve detection accuracy and robustness.
- Dynamic threshold adjustment based on vehicle speed, lane proximity, or traffic density.
- Improved fault detection and recovery in the case of thread failure or communication loss.

These advancements would further enhance the BSD module's reliability and effectiveness, reinforcing its role in improving situational awareness and lateral safety in modern intelligent vehicles.

4.5 Traffic Sign and Traffic Light Recognition System

4.5.1 Introduction

The development of an autonomous vehicle requires a robust perception system capable of understanding and interpreting traffic signs and traffic lights in real-time. This chapter presents the implementation of a comprehensive traffic sign and traffic light recognition system that serves as a critical component of our Self-Driving Vehicle (SDV) project. The system employs advanced computer vision techniques to identify, classify, and communicate traffic information to the vehicle's control systems.

4.5.2 System Architecture Overview

The traffic recognition system consists of several interconnected components working in harmony to provide reliable detection and classification capabilities:

- **Deep Learning Model:** YOLOv8 Nano for object detection and classification
- **Model Optimization:** ONNX conversion for enhanced performance
- **Communication Protocol:** MQTT for real-time data transmission
- **User Interface:** GUI for visualization and monitoring
- **Frame Comparison Algorithm:** Intelligent filtering to reduce redundant outputs
- **Camera Control System:** Dynamic detection activation/deactivation

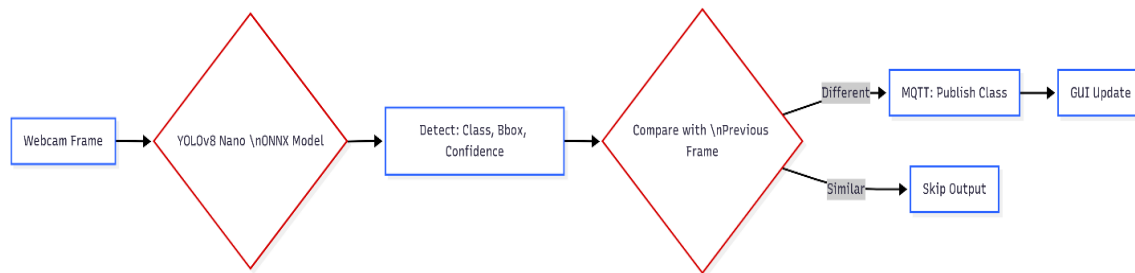


Figure 4.5.1: Flowchart representing the internal logic of the Traffic Sign Recognition (TSR)

4.5.3 Object Detection Model Implementation

I. YOLOv8 Nano Selection

For this project, we implemented YOLOv8 Nano as our primary object detection model. The selection of YOLOv8 Nano was based on several critical factors:

- **Performance Efficiency:** YOLOv8 Nano provides an optimal balance between accuracy and computational efficiency, making it suitable for real-time applications in autonomous vehicles
- **Resource Optimization:** The nano variant requires minimal computational resources while maintaining high detection accuracy
- **Real-time Processing:** Capable of processing video frames at sufficient frame rates for autonomous driving applications

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Figure 4.5.2: YOLO Versions' Performance Metrics

II. Model Performance Metrics

Our trained YOLOv8 Nano model achieved exceptional performance metrics:

- **Mean Average Precision (mAP):** 97.5%
- **Detection Classes:** 15 distinct traffic signs and traffic light states
- **Inference Speed:** Optimized for real-time processing

The high mAP score of 97.5% demonstrates the model's reliability in accurately detecting and classifying traffic signs and traffic lights under various

environmental conditions.

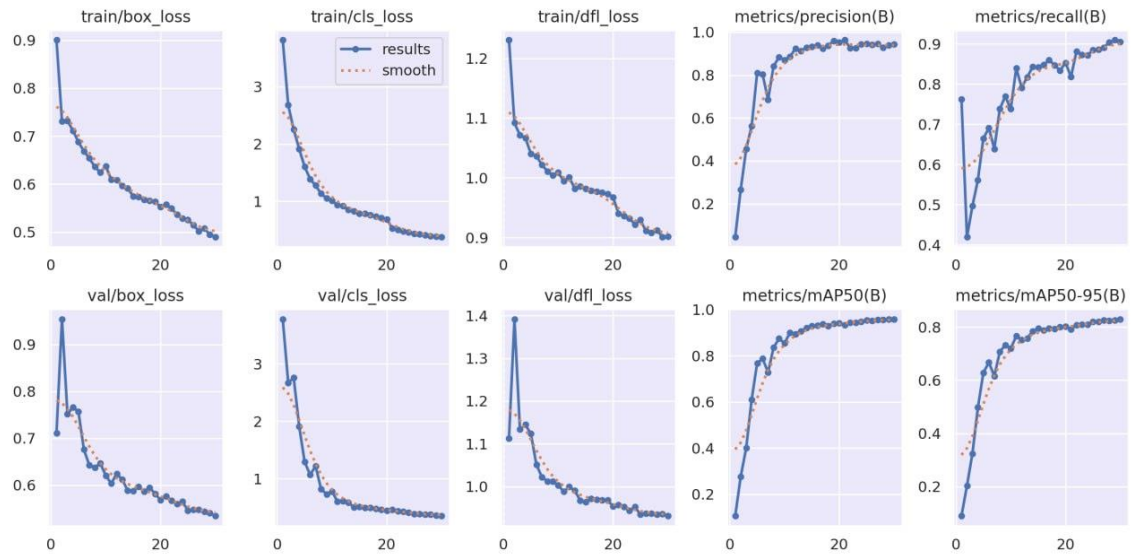


Figure 4.5.3: Analysis of Loss Functions and Recall Metrics



Figure 4.5.4: Model Output

4.5.4 Model Conversion: PyTorch to ONNX

I. Conversion Rationale

The conversion of our trained model from PyTorch (.pt) format to Open Neural Network Exchange (ONNX) format was implemented for several strategic reasons:

- **Cross-Platform Compatibility:** ONNX provides a universal format that enables model deployment across different hardware platforms and

software frameworks, ensuring flexibility in our SDV system architecture.

- **Performance Optimization:** ONNX models typically offer improved inference speed due to optimized runtime engines and graph-level optimizations that reduce computational overhead.
- **Memory Efficiency:** The ONNX format often results in smaller model file sizes and reduced memory footprint during inference, which is crucial for embedded systems in autonomous vehicles.
- **Production Deployment:** ONNX models are better suited for production environments, offering more stable and predictable performance characteristics compared to framework-specific formats.

II. Conversion Process

The conversion process involved:

- Loading the trained YOLOv8 Nano model in PyTorch format
- Exporting the model to ONNX format using the built-in export functionality
- Validating the converted model to ensure performance consistency
- Optimizing the ONNX model for target hardware specifications

4.5.5 Classification System

I. Traffic Sign and Light Classes

Our system recognizes 15 distinct classes of traffic signs and traffic light states:

Class ID	Classification	Description
0	Green Light	Traffic light indicating proceed
1	Red Light	Traffic light indicating stop
2	Speed Limit 10	Speed restriction: 10 km/h
3	Speed Limit 100	Speed restriction: 100 km/h
4	Speed Limit 110	Speed restriction: 110 km/h
5	Speed Limit 120	Speed restriction: 120 km/h
6	Speed Limit 20	Speed restriction: 20 km/h
7	Speed Limit 30	Speed restriction: 30 km/h
8	Speed Limit 40	Speed restriction: 40 km/h
9	Speed Limit 50	Speed restriction: 50 km/h
10	Speed Limit 60	Speed restriction: 60 km/h
11	Speed Limit 70	Speed restriction: 70 km/h
12	Speed Limit 80	Speed restriction: 80 km/h
13	Speed Limit 90	Speed restriction: 90 km/h
14	Stop	Stop sign requiring complete halt

II. Class Mapping Implementation

The system utilizes a comprehensive class mapping dictionary that translates numerical class IDs to human-readable descriptions. This mapping facilitates:

- **GUI Display:** Clear presentation of detected signs to system operators
- **System Integration:** Seamless communication with other SDV subsystems
- **Data Logging:** Structured recording of detection events for analysis
- **Debugging:** Simplified troubleshooting and system monitoring

4.5.6 Communication Protocol: MQTT Integration

I. MQTT Implementation

The system employs Message Queuing Telemetry Transport (MQTT) protocol for real-time communication between the detection system and the graphical user interface. MQTT was selected for the following advantages:

- **Low Latency:** Minimal communication delay for real-time applications
- **Lightweight Protocol:** Efficient bandwidth utilization
- **Reliability:** Built-in quality of service (QoS) levels ensure message delivery
- **Scalability:** Easy integration with multiple system components

II. Data Transmission Format

Detection results are transmitted via MQTT in structured format containing:

- **Class Name:** Human-readable description of detected object
- **Confidence Score:** Detection probability percentage
- **Bounding Box Coordinates:** Spatial location of detected object
- **Timestamp:** Detection time for synchronization
- **Frame ID:** Unique identifier for frame correlation

4.5.7 Enhanced Frame Comparison Algorithm

I. Intelligent Output Filtering

To optimize system performance and reduce redundant communications, we implemented an advanced frame comparison algorithm that analyzes consecutive frames before transmitting detection results.

II. Comparison Parameters

The algorithm evaluates three key elements between the current frame and the previous frame:

- **Class Name Comparison:**
 - Identifies whether the same traffic sign or traffic light class is detected
 - Prevents repeated transmission of identical classifications
- **Bounding Box Analysis:**
 - Compares spatial coordinates and dimensions of detected objects
 - Accounts for minor variations due to camera movement or object perspective changes
 - Implements tolerance thresholds to handle natural fluctuations
- **Object Presence Detection:**
 - Determines if the same object continues to be present in the scene
 - Tracks object persistence across multiple frames
 - Identifies new objects entering the detection area

III. Decision Logic

The system employs the following decision logic:

```
IF (current_class == previous_class) AND
   (bounding_box_similarity > threshold) AND
   (object_present == TRUE)
THEN
    suppress_output()
ELSE
    transmit_detection_result()
```

This approach significantly reduces:

- **Network Traffic:** Minimizes MQTT message frequency
- **GUI Updates:** Prevents unnecessary interface refreshes
- **System Load:** Reduces processing overhead
- **Data Redundancy:** Eliminates duplicate detection logs

4.5.5 Camera Control System

I. Dynamic Detection Control

The system incorporates intelligent camera control functionality that allows for dynamic activation and deactivation of the detection process. This feature provides:

- **Power Management:**
 - Reduces computational load when detection is not required
 - Extends system battery life in mobile applications
 - Optimizes resource allocation
- **Operational Flexibility:**
 - Enables manual override of automatic detection
 - Supports different operational modes (training, testing, production)
 - Allows for system maintenance without complete shutdown
- **Performance Optimization:**
 - Prevents unnecessary processing during idle periods
 - Maintains camera feed while suspending detection algorithms
 - Enables quick reactivation when needed

4.5.9 Future Enhancements

I. Planned Improvements

Several enhancements are planned for future system iterations:

- **Multi-Camera Integration:** Support for multiple camera inputs
- **3D Object Detection:** Enhanced spatial understanding
- **Temporal Tracking:** Object tracking across multiple frames
- **Adaptive Thresholding:** Dynamic adjustment of detection parameters

II. Scalability Considerations

The system architecture supports future scalability through:

- **Modular Design:** Easy addition of new detection classes
- **API Integration:** Support for external system connections
- **Cloud Connectivity:** Potential for cloud-based processing
- **Edge Computing:** Deployment on edge devices for distributed processing

4.5.10 Conclusion

The implemented traffic sign and traffic light recognition system represents a sophisticated solution for autonomous vehicle perception. Through the integration of YOLOv8 Nano, ONNX optimization, MQTT communication, and intelligent frame comparison algorithms, the system achieves high accuracy (97.5% mAP) while maintaining real-time performance. The dynamic camera control functionality and robust classification system ensure reliable operation across diverse driving conditions, making it a valuable component of our Self-Driving Vehicle project.

The system's modular architecture and comprehensive feature set provide a solid foundation for future enhancements and integration with additional autonomous driving components. The successful implementation demonstrates the viability of using advanced computer vision techniques for critical safety applications in autonomous vehicles.

4.6: Linux Image

4.6.1 Overview

This chapter explains the software infrastructure that connects the lower-level embedded subsystems with the high-level ADAS features. The embedded Linux image on the Raspberry Pi provides the runtime environment for all decision-making modules, which are deployed in isolated containers. These containers process sensor data, run AI models, and communicate using MQTT to coordinate system behavior.

To support the integration of ADAS features (Blind Spot Detection, Emergency Braking System, Traffic Sign Recognition), a custom Embedded Linux image was built using the Eclipse Leda framework. This Yocto-based distribution is tailored for Software-Defined Vehicles (SDVs), providing a modular, lightweight, and container-ready OS.

4.6.2 Embedded Linux

Embedded Linux refers to the use of the standard Linux kernel together with a tailored set of libraries and utilities on devices with specific functions and constraints. In contrast to desktop Linux, an embedded Linux system is optimized for reliability, limited hardware resources (CPU, memory, power), and long-term support.

Its open-source nature and flexibility make it a popular choice for countless devices – from consumer electronics to industrial controllers – and it is widely adopted in automotive systems for running in-vehicle software. In essence, embedded Linux provides a stable OS platform that can be customized to meet the needs of an embedded application while leveraging the robustness and community support of the Linux ecosystem.

4.6.3 Yocto Project

The Yocto Project is an open-source collaboration initiative that provides tools and frameworks for creating custom Linux-based operating systems for embedded devices.

Rather than being a Linux distribution itself, Yocto supplies a build system (based on OpenEmbedded and BitBake) along with a collection of metadata "recipes" that allow developers to generate a tailored Linux distribution for their hardware. Yocto's goal is to enable the rapid, repeatable development of embedded Linux images independent of the underlying CPU architecture.

Using Yocto, engineers can define exactly which kernel, device drivers, libraries, and applications are included in the OS image, ensuring the resulting system meets the project's requirements.

4.6.4 Containers in Automotive Software

I. What Are Containers?

Containers are a modern software technology used to package an application along with everything it needs to run—such as libraries, dependencies, configuration files, and binaries—into a single, self-contained unit. This unit can then run reliably across different computing environments, whether it's on a developer's laptop, a server, or an embedded device.

Unlike traditional applications that rely on the host system's installed software, a container carries its own environment. It shares the host's operating system kernel but runs in an isolated user-space, meaning it has its own filesystem, networking, and process tree. This approach makes containers much more lightweight and faster to start than virtual machines, while still ensuring consistent behavior across platforms.

Popular container technologies include Docker and containerd, and they've become widely used in software development because they:

- Ensure consistency across development, testing, and production
- Simplify deployment and rollback
- Improve modularity and reusability of code

II. Why Containers Are Used in Software Projects

In general software engineering, containers offer many advantages:

- **Isolation:** Each application runs in its own container, preventing conflicts.
- **Portability:** A container will run the same way on any compatible system.
- **Scalability:** Containers can be deployed and scaled independently.
- **Efficiency:** They use fewer resources than virtual machines.

These benefits make containers ideal for complex, multi-module projects where components are developed and maintained independently. They're also well-suited for CI/CD pipelines and cloud-based deployment.

III. Why Containers Are Ideal for Automotive and ADAS

In the automotive context—especially for Software-Defined Vehicles (SDVs) and Advanced Driver Assistance Systems (ADAS)—containers offer critical advantages:

- **Modular Design:** Each ADAS feature (e.g., BSD, EBS, TSR) can be developed, updated, and tested in isolation.
- **Safety and Fault Tolerance:** If one container crashes (e.g., due to a bug in TSR), others (like EBS) continue running unaffected.
- **Over-the-Air Updates:** Containers allow targeted OTA updates for a specific module without touching the whole system.
- **Hardware Independence:** A containerized TSR module developed on a PC can be directly deployed to a Raspberry Pi running Leda with little or no change.
- **Cleaner Integration:** MQTT-based communication between containers ensures loose coupling and real-time data flow.

In this project, containers were used to implement each ADAS module independently, ensuring modularity, easier debugging, and safe deployment. Their use also laid the foundation for a production-like SDV architecture, where software functions are decoupled from hardware and can evolve continuously.

4.6.5 Eclipse Leda

I. What is Eclipse Leda?

Eclipse Leda is an open-source embedded Linux distribution designed specifically for Software-Defined Vehicles (SDVs). It builds on the Yocto Project and provides a complete, production-grade framework for deploying automotive software on modern hardware.

Leda integrates essential components such as Linux kernel recipes, container runtimes, OTA mechanisms, and middleware for SDV use cases. It simplifies the development of in-vehicle software by offering a ready-made, modular Linux image that supports containerized applications and over-the-air updates.

II. Why Eclipse Leda in This Project?

Eclipse Leda was chosen for this project due to its SDV-oriented features that align directly with ADAS system requirements:

- **Container-based deployment:** Each ADAS function (Blind Spot Detection, Emergency Braking, Traffic Sign Recognition, and Communication App) is packaged and deployed as a separate container. This improves isolation, modularity, and maintainability.
- **OTA support:** Leda provides integrated Over-the-Air update capabilities, allowing software updates to be delivered remotely — including container updates — without disrupting the entire system.
- **Yocto-based extensibility:** Since it is built with Yocto, Eclipse Leda is customizable, enabling the inclusion of only the components necessary for this ADAS use case.
- **Edge-ready:** Optimized to run on platforms like Raspberry Pi or automotive-grade hardware, it allows for easy prototyping and scaling.

III. Container Management in Practice

One of the key advantages of Eclipse Leda is its native support for Over-the-Air (OTA) updates, which enables secure and remote software delivery to the vehicle. In this project, the OTA mechanism is leveraged to update individual application containers rather than the entire system image. This means that each ADAS module (e.g., BSD, EBS, TSR) can be independently updated by simply replacing its container image.

When a new version of a module is available (for example, an improved traffic sign recognition model), it is packaged as a new container image and uploaded to a remote container registry. The Eclipse Leda system fetches the new image and seamlessly replaces the running container with the updated one — all without requiring a full system reboot or flashing the entire OS.

This modular update approach significantly reduces system downtime, enhances development flexibility, and allows for faster iteration cycles, especially during testing or field deployment. The update process is secure, version-controlled, and rollback-capable, ensuring system stability even if an update fails.

IV . OTA Workflow Overview

One of the key advantages of Eclipse Leda is its native support for Over-the-Air (OTA) updates, which enables secure and remote software delivery to the vehicle. In this project, the OTA mechanism is leveraged to update individual **application containers** rather than the entire system image. This means that each ADAS module (e.g., BSD, EBS, TSR) can be independently updated by simply replacing its container image.

When a new version of a module is available (for example, an improved traffic sign recognition model), it is packaged as a new container image and uploaded to a remote container registry. The Eclipse Leda system fetches the new image and seamlessly replaces the running container with the updated one — all **without requiring a full system reboot or flashing the entire OS**. This modular update approach significantly **reduces system downtime**, enhances development flexibility, and allows for **faster iteration cycles**, especially during testing or field deployment.

The update process is secure, version-controlled, and rollback-capable, ensuring system stability even if an update fails.

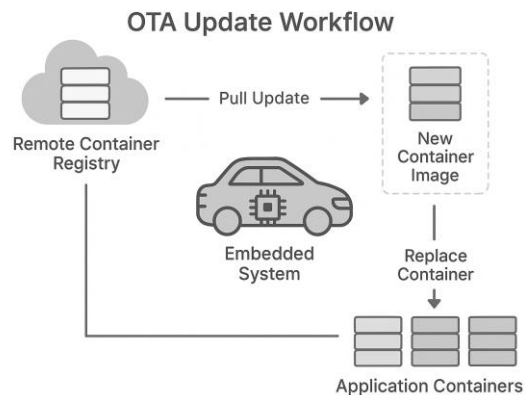
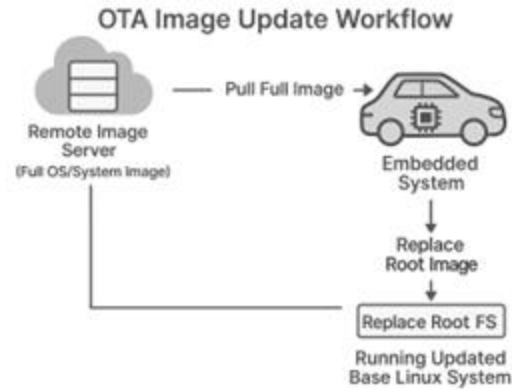


Figure 4.6.1: Over-the-Air (OTA) Container Update Workflow for Embedded Systems



4.6.2: Over-the-Air (OTA) Image Update Workflow for Embedded Systems

4.6.6 Notes on Implementation Scope

In this project, each ADAS function — including Blind Spot Detection, Emergency Braking, and Traffic Sign Recognition — was developed as a standalone module packaged within its own container. These containers were manually constructed, with careful attention to bundling all required dependencies, such as libraries, AI model runtimes (e.g., ONNX), and communication interfaces. This approach allowed each module to operate independently, with complete control over its runtime environment.

Container deployment was handled using **Eclipse Kanto**, the lightweight container management system integrated into Eclipse Leda. For each module, a dedicated **manifest file** was written to define:

- The container image and version.
- Access permissions to hardware devices (e.g., SPI interface or camera input).
- Network requirements, including access to the MQTT broker.
- Runtime configurations, such as environment variables, volume mounts, and restart policies.

Kanto provided a clean interface to launch, update, and stop containers without the need for low-level scripting or orchestration. It also ensured that each container was isolated, secure, and could be restarted independently in case of failure or during development.

The containers communicated through a structured **MQTT-based messaging layer**, which served as the core of the system's internal communication. A central communication application (running outside the containerized stack) connected to the STM32 microcontroller and published sensor data to MQTT topics such as

- sensors/front (for EBS)
- sensors/left and sensors/right (for BSD)

The ADAS containers subscribed to these topics to receive real-time input. Based on this input, each module published alerts to topics like info/ebs, info/bsd/left, info/bsd/right, and info/tsr, which were then consumed by other system components.

To support driver interaction and provide system feedback, a dedicated **infotainment application** was developed using C++ and Qt. Which was fully integrated into the SDV architecture. It subscribed to the relevant MQTT topics to:

- Display ADAS alerts and recognized traffic signs.
- Provide the driver with toggles to enable or disable modules via control topics (control/ebs, control/bsd, control/tsr).
- Issue a startup signal via the actuators topic when the system was activated.

The infotainment system played a key role in human-machine interaction, offering a visual interface to monitor and manage the ADAS modules in real-time.

This modular, container-based design enabled efficient development, easy debugging, and seamless OTA integration. By separating each module into its own container and using MQTT as a decoupled communication layer, the project followed SDV best practices and delivered a flexible, scalable prototype aligned with modern automotive software architecture.

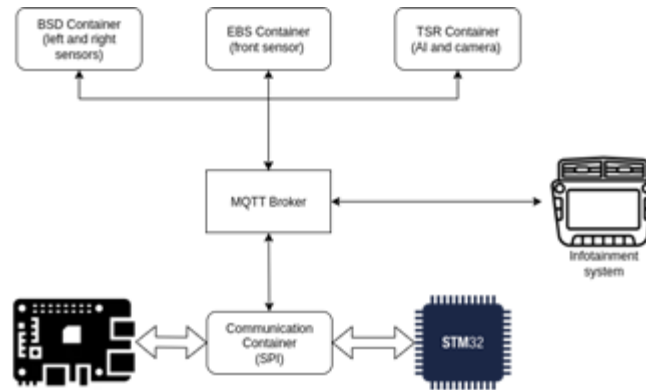


Figure 4.7.3: ADAS Container Architecture and Communication Workflow in Eclipse Leda

4.6.7 Testing and Validation of Containers

Each container developed for the ADAS modules was tested both individually and as part of the complete system.

Unit testing was performed by feeding mock MQTT messages to the relevant containers and verifying their expected outputs. For instance, the BSD and EBS containers were tested using simulated ultrasonic readings to confirm that the status messages (e.g., “Warning” or “Danger”) were published to the correct MQTT topics. The Traffic Sign Recognition (TSR) container was tested using pre-recorded image sequences to validate the inference results and ensure correct integration with the MQTT messaging layer.

Integration testing involved connecting the actual STM32 microcontroller and the camera to the Raspberry Pi running Eclipse Leda. Real-time sensor data from the hardware was published via the communication application, and the runtime behavior of each container was observed. Logging mechanisms (like spdlog) and MQTT clients (such as MQTT Explorer) were used to monitor traffic, debug issues, and confirm correct communication between modules.

4.6.8 Security Considerations

Security was addressed during the design and deployment of the containerized ADAS modules. Each container was assigned **only the necessary permissions** through its manifest file. For instance, a container that required access to a camera or SPI interface was granted limited access to those specific devices via `/dev/`.

All containers were isolated using standard Linux namespaces, preventing direct access to host resources or other containers.

The **MQTT broker** used for inter-container communication was configured for **local access only**, which restricted any remote access and reduced the attack surface. While advanced features like container image signing and encrypted MQTT transport (TLS) were not implemented in this prototype, the platform (Eclipse Leda) provides support for secure OTA delivery, which could be integrated in future versions for enhanced protection.

4.6.9 Summary

This chapter presented the embedded software infrastructure that powers the ADAS modules of the project. A custom Embedded Linux image was built using the Eclipse Leda framework, combining the flexibility of the Yocto Project with built-in support for containers and over-the-air updates.

Each ADAS function—Blind Spot Detection, Emergency Braking System, and Traffic Sign Recognition—was developed and deployed as an independent container. These containers were manually configured with their required dependencies and connected through a structured MQTT messaging system. Deployment and lifecycle management were handled using Eclipse Kanto, with detailed manifest files specifying hardware access, network policies, and runtime behavior.

The system was tested both modularly and as an integrated platform. OTA update mechanisms allowed for seamless container-level updates without needing to reboot the entire system. Additionally, an infotainment application was integrated to provide real-time visual feedback and control over the ADAS modules.

By combining embedded Linux, containerization, modular design, and MQTT-based communication, the project successfully delivered a scalable, maintainable, and modern prototype aligned with Software-Defined Vehicle (SDV) principles.

4.7: INFOTAINMENT SYSTEM

4.7.1 Introduction

I. Role of the infotainment system

The infotainment system serves as the primary interface between the driver and the Software-Defined Vehicle (SDV) environment. It provides centralized access to critical safety information and feature control. Its core functions include:

- Starting the vehicle.
- Display real-time safety notifications for:
 - Emergency Braking System (EBS).
 - Blind Spot Detection (BSD).
- Presenting recognized traffic signs from the Traffic Sign Recognition (TSR) module.
- Allowing the driver to enable or disable specific ADAS features (e.g., turning off BSD if not needed).

II. Design Goals

The system was designed with the following objectives:

- Real-Time Responsiveness: Immediate notification of critical driving events.
- Ease of Control: Intuitive toggling of safety modules.
- Modularity: Decoupled architecture allowing independent development and testing of system modules.

III. Data Flow

The infotainment system acts as both data sink (displaying system status) and a control source (issuing user commands) within the vehicle's architecture:

- Inputs:
 - ADAS status updates from EBS, BSD, and TSR modules.
- Outputs:
 - Visual warnings displayed to the driver & control commands.
 - Control commands.

4.7.2 System Design

I. System Architecture

- The infotainment system is designed as a modular application that interfaces with the MQTT-based communication layer of the SDV.

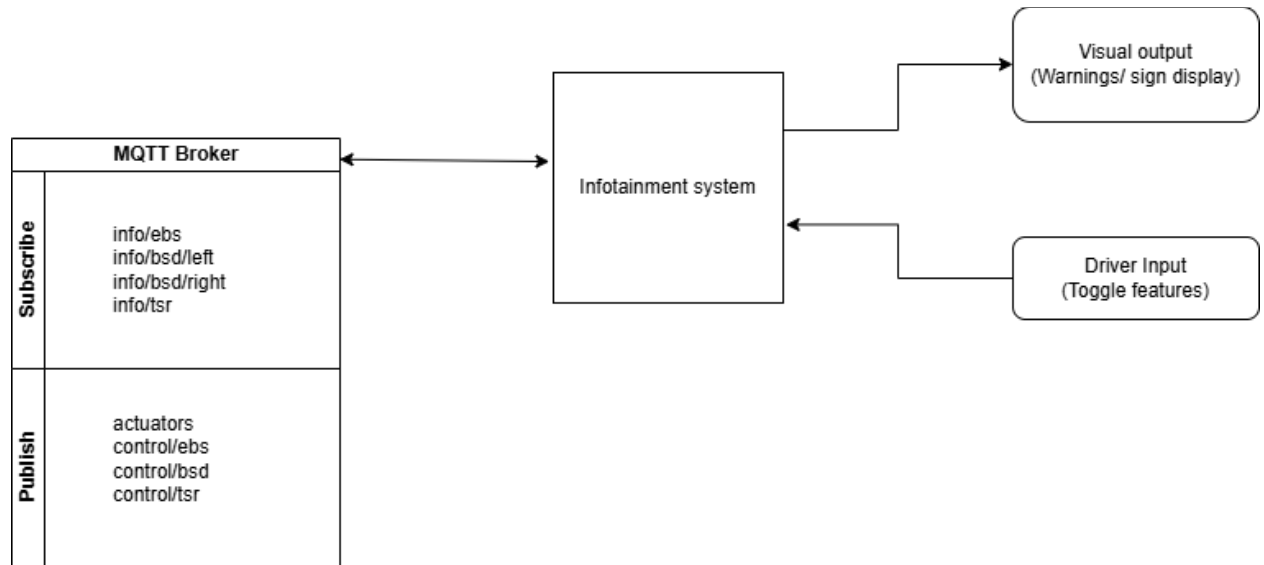


Figure 4.7.1: Infotainment System block diagram

II. Functional Components

a. Communication Module

- Manages all MQTT operations (Connection, subscriptions, and publishing).
- Subscribed topics include:
 - `info/ebs`: Emergency braking status.
 - `info/bsd/left` and `info/bsd/right`: Blind spots status.
 - `info/tsr`: recognized traffic signs.
- Published topics include:
 - `control/ebs`, `control/bsd`, and `control/tsr`: Used to enable/disable ADAS features.
 - `actuators`: Used to start the vehicle.

b. User Interface (GUI)

- i. Provides a start button to start the vehicle.
- ii. Display alerts and signs with visual indicators (images, color-coded panels, etc.).
- iii. Provides control buttons/switches for enabling/disabling modules.
- iv. Designed using Qt and C++.

c. Control Logic

- i. Ensures UI feedback is updated according to received statuses.
- ii. Listens for driver inputs and translates them into MQTT messages.

III. Data Flow Summary

- Driver command to start the vehicle is sent to the vehicle's actuators.
- Status updates received from ADAS modules are displayed to the driver.
- Driver control commands (enable/disable) are sent to the ADAS modules.

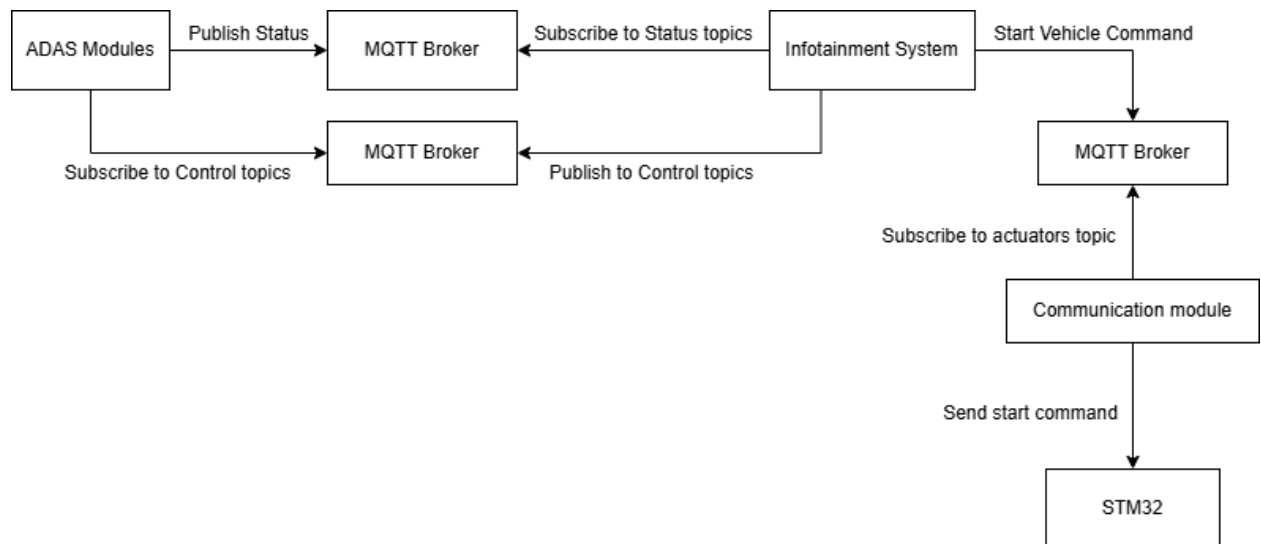


Figure 4.7.2: Data flow within the system

4.7.3 Implementation

I. Software Architecture

The infotainment system was implemented in C++ using Qt framework, following a modular architecture to ensure separation of concerns and ease of maintainability. The main components include:

- **Communication Module:**
 - Manages MQTT connectivity with the system's broker
 - Handles topic subscriptions and message publishing.
- **ADAS Notifications Module:**
 - Subscribes to ADAS status topics:
 - info/ebs
 - info/bsd/left
 - info/bsd/right
 - info/tsr
 - Processes incoming messages and emits corresponding Qt signals to the UI layer
- **UI Modules:**
 - Start Window:
 - Display the system's startup interface.
 - Allows the user to initiate the vehicle startup.
 - Publishes a control signal on the actuators topic.
 - Main Window:
 - Displays ADAS notifications and traffic signs updates
 - Provides toggles to control:
 - EBS (control/ebs)
 - BSD (control/bsd)
 - TSR (control/tsr)

This modular structure promotes a clean separation between network communication, data processing, and user interactions, enabling future enhancements with minimal redesign.

II. Data Flow & Control Logic

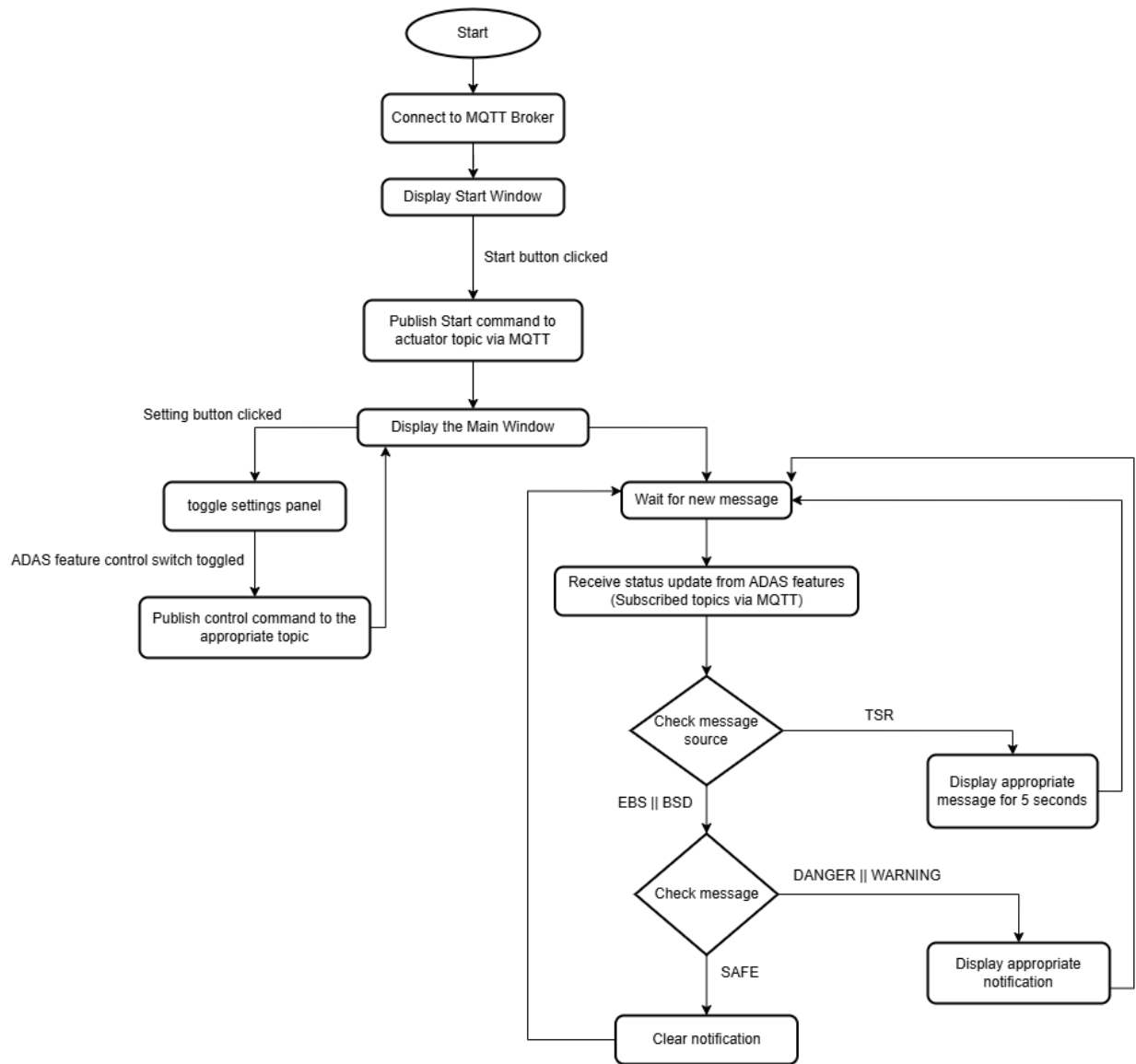


Figure 4.7.3: Infotainment system control flow.

This flowchart illustrates the runtime behavior of the infotainment system, starting from MQTT connection, vehicle startup, through GUI interactions and real-time ADAS updates.

4.7.4 GUI Implementation

I. Start Window

The start window serves as the initial interface displayed when the infotainment system is launched. It features a minimalist design to reduce distractions and focus on the primary action: starting the vehicle. The window consists of a plain background with a centrally aligned “START” button, which the driver must press to initiate the system. Upon clicking the button, a control command is published to the MQTT broker on actuators topic, signaling the system to begin normal operations. After this the interface transitions to the main window. This window ensures clear entry point into the system, preventing premature interaction with safety critical features before system initialization.

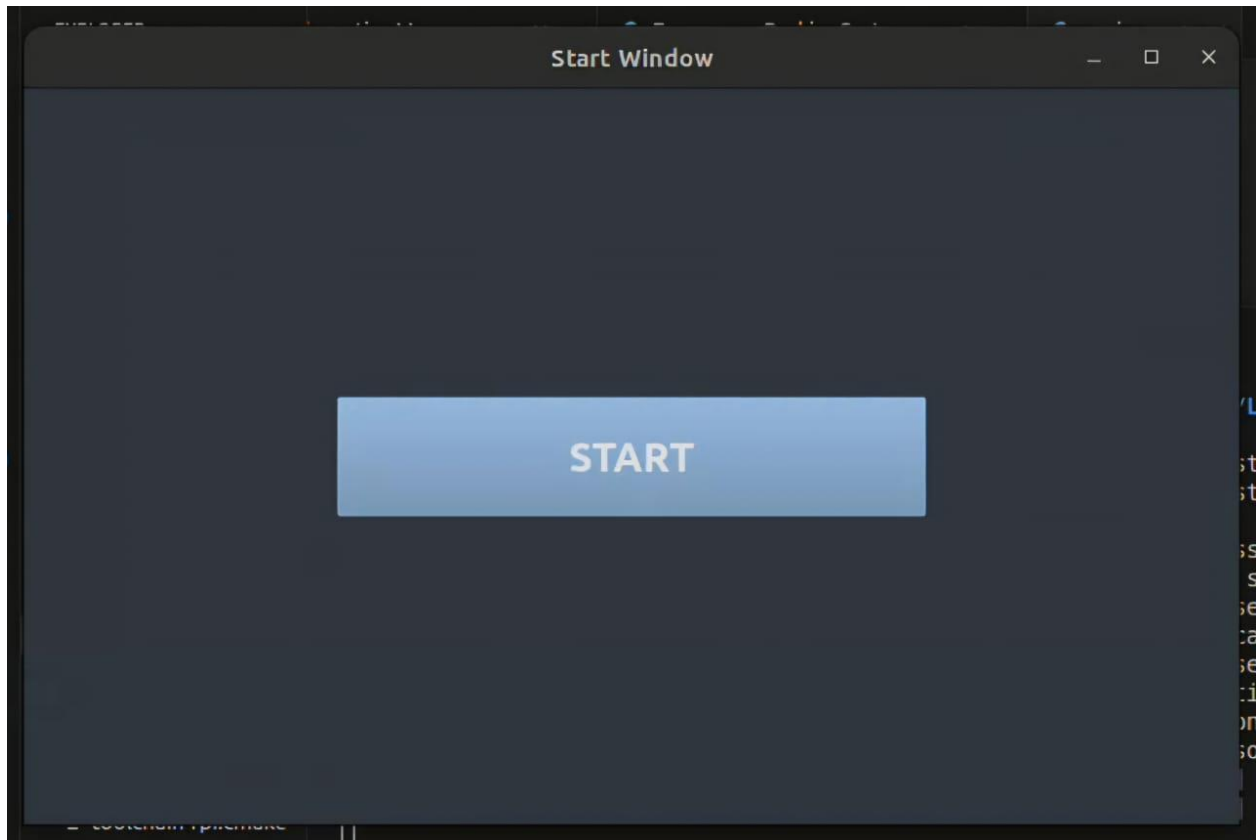


Figure 4.7.4: Start Window of the infotainment system

A simple interface showing the centrally positioned start button, used to initiate system activation via MQTT.

II . Main Window

The Main Window is the primary interface shown after the vehicle is started. Its main purpose is to display real-time driver notifications received from the ADAS modules. These alerts include Emergency Braking System (EBS), Blind Spot Detection (BSD), and recognized traffic signs via Traffic Signs Recognition (TSR). Warnings are displayed using color-coded labels and clear text to indicate the severity of each alert (e.g., “DANGER” “WARNING”). This layout ensures that the driver’s attention is drawn immediately to critical updates without unnecessary clutter.

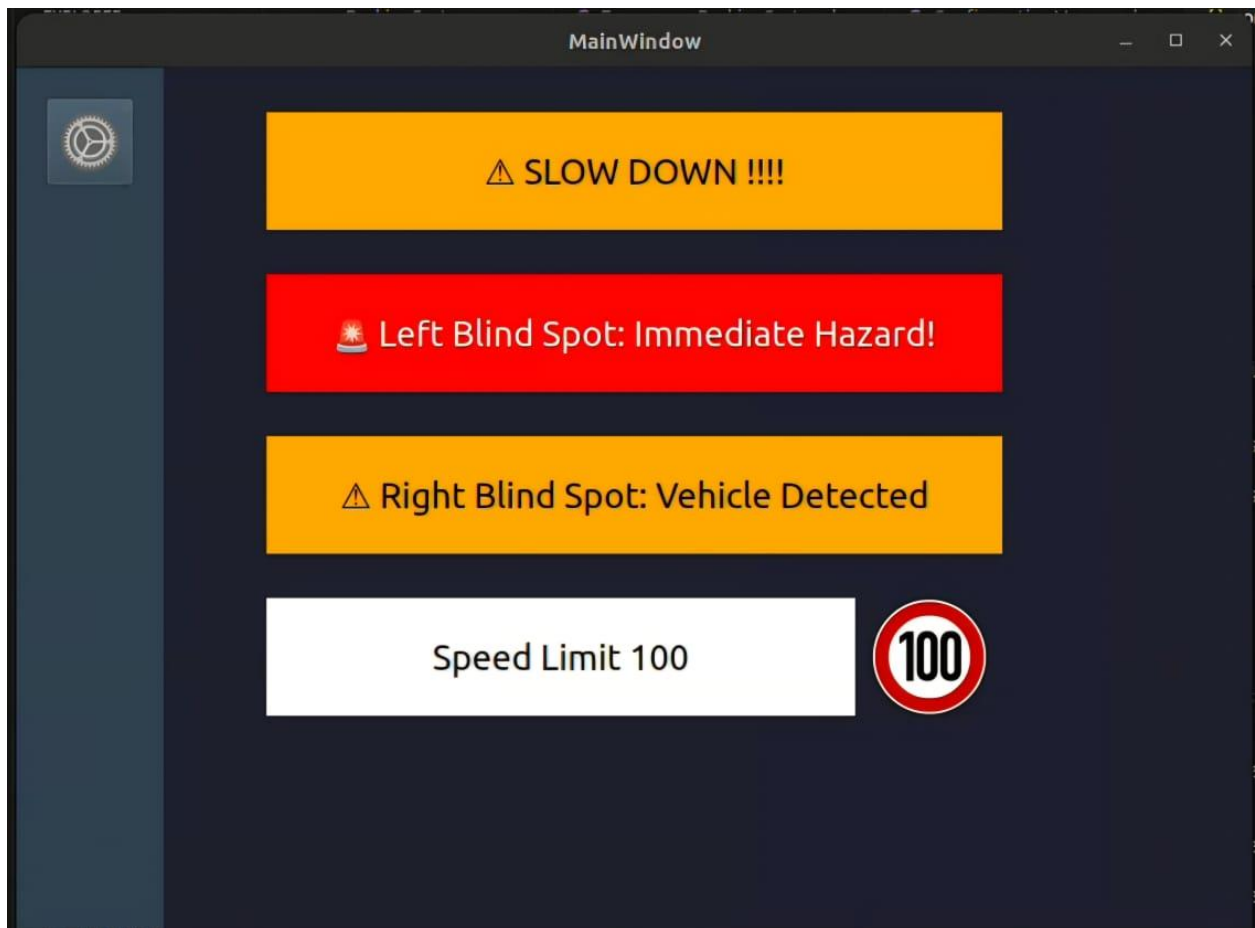


Figure 4.7.5: Main Window displaying ADAS notifications

A sample runtime state where warnings are displayed in real time

To maintain a clean interface while still allowing user control, the Main Window includes a side panel located at the left edge of the screen containing a settings button. When the setting button is pressed a panel slides out containing individual toggles for each ADAS feature, allowing the driver to enable or disable modules such as EBS, BSD, or TSR as desired. Pressing the settings button again closes the panel. This design offers quick access to safety system configuration without interrupting the main display area. Any changes made through this panel are immediately published as MQTT control messages to the relevant topics.

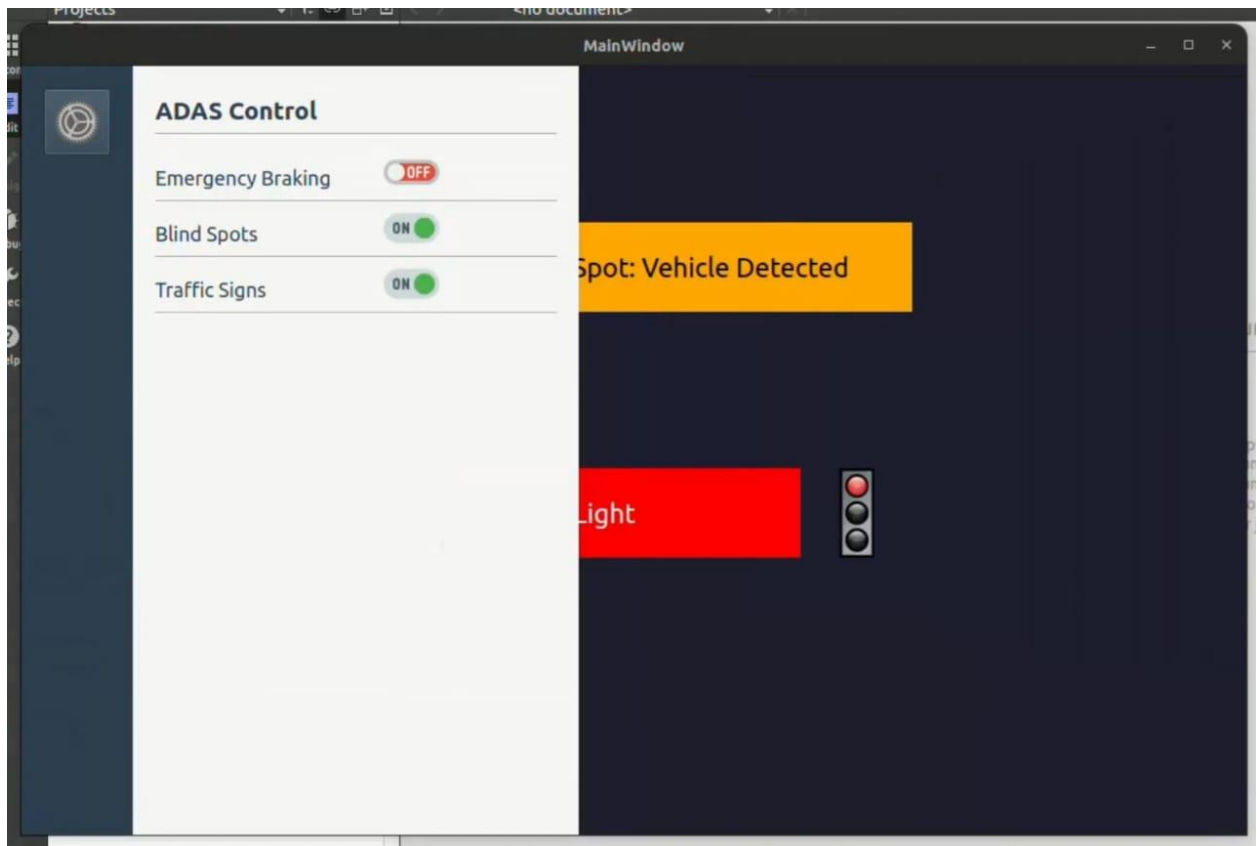


Figure 4.7.6: Main Window with sliding settings panel open

The settings panel is shown extended, revealing ADAS control switches.

4.7.5 Summary

The infotainment system serves as a crucial interface between the driver and the underlying ADAS modules of the software-defined vehicle. This chapter presented the system's design and implementation, highlighting its modular architecture, real-time communication via MQTT, and user-oriented interface. Through its Start Window, the system ensures a deliberate and safe activation process, while the Main Window provides both live status feedback and intuitive control over ADAS features. The combination of persistent warnings, time-bound traffic sign displays, and a sliding settings panel ensures both clarity and responsiveness. The system's design meets its core goals of real-time awareness, ease of use, and modularity, laying the groundwork for future expansion and enhancement of in-vehicle interaction systems.

4.7.6 Future Work

While the current implementation fulfills its primary objectives, several enhancements can be introduced in future iterations. Some potential improvements are:

- Integrate auditory alerts
- Implement a logging system to record incoming data and driver's interactions for:
 - Performance evaluation
 - System diagnostics
 - Debugging

Finally introducing over-the-air (OTA) update capabilities would enable remote software maintenance and feature upgrade without requiring physical access to the system.

CHAPTER 5: Experimental Extension: Lane Keeping Assist (LKA)

5.1 Introduction

The Lane Keeping Assist (LKA) system represents a fundamental component of modern autonomous vehicle technology, designed to maintain vehicle positioning within lane boundaries through real-time lane detection and steering guidance. This chapter presents the implementation of a comprehensive LKA system that serves as a critical safety feature in our Self-Driving Vehicle (SDV) project. The system employs advanced computer vision techniques combined with edge detection algorithms to identify lane markings and provide precise steering commands to maintain optimal vehicle trajectory.

The LKA system operates as an active safety feature that continuously monitors the vehicle's position relative to lane boundaries and provides corrective steering input when lane departure is detected. This implementation utilizes OpenCV for image processing, MQTT protocol for communication, and real-time video analysis to ensure reliable lane detection under various driving conditions.

The Lane Keeping Assist feature has been successfully developed and tested independently. However, it is not yet integrated into the main system. Integration is planned for a future update to ensure seamless interaction with other vehicle control components.

5.2 System Architecture Overview

The Lane Keeping Assist system consists of several interconnected components working in harmony to provide reliable lane detection and steering guidance capabilities:

- **Computer Vision Engine:** OpenCV-based image processing for lane detection
- **Edge Detection Algorithm:** Canny edge detection for lane boundary identification
- **Region of Interest (ROI) Processing:** Focused analysis on relevant road areas
- **Hough Transform:** Line detection and lane boundary extraction

- **Steering Angle Calculation:** Mathematical computation of required steering corrections
- **Communication Protocol:** MQTT for real-time command and control
- **Dynamic Control System:** Remote activation/deactivation capability
- **Real-time Visualization:** Live feedback display with overlay information

5.3 Computer Vision Implementation

5.3.1 Image Preprocessing Pipeline

The LKA system implements a sophisticated image preprocessing pipeline designed to enhance lane detection accuracy under various environmental conditions:

Grayscale Conversion: The system converts RGB color frames to grayscale to reduce computational complexity while preserving essential edge information necessary for lane detection. This conversion simplifies subsequent processing steps and improves algorithm efficiency.

Gaussian Blur Application: A 5×5 Gaussian kernel is applied to reduce image noise and smooth pixel intensity variations. This preprocessing step helps eliminate minor imperfections and improves the reliability of edge detection algorithms.

Canny Edge Detection: The system employs Canny edge detection with optimized threshold values (50-150) to identify lane boundary edges. The dual-threshold approach ensures detection of both strong and weak edges while maintaining noise immunity.

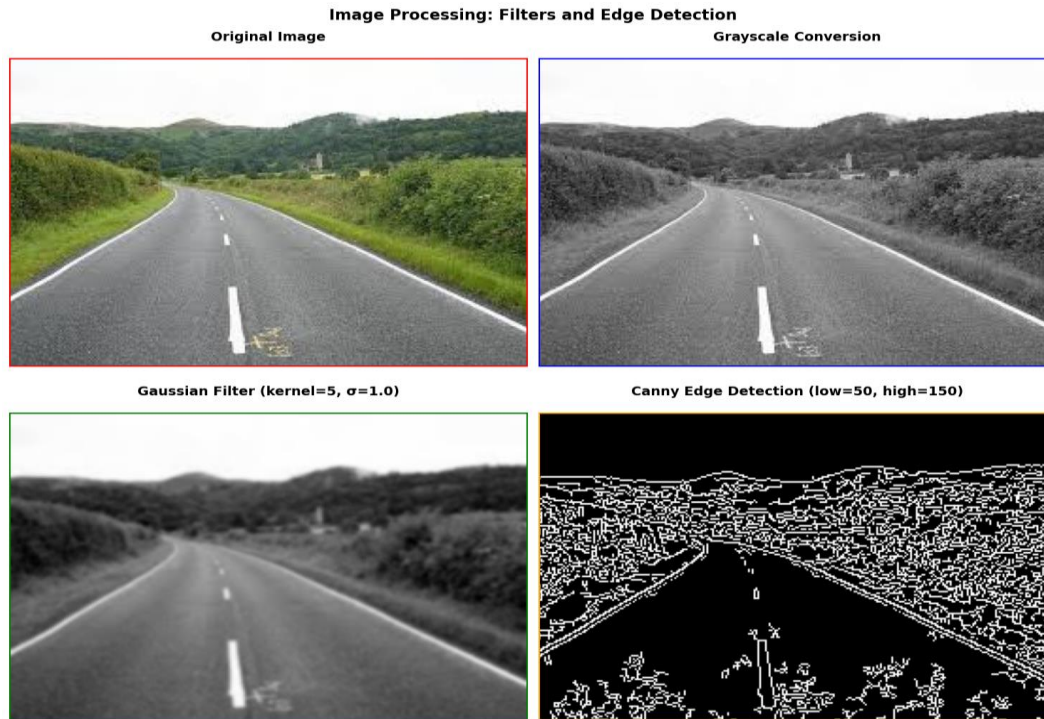


Figure 5.2: Visualizing Image Processing Techniques: Filters and Edge Detection

5.3.2 Region of Interest (ROI) Definition

To optimize processing efficiency and focus on relevant road areas, the system implements an intelligent ROI selection algorithm:

Trapezoidal ROI Mask: The system defines a trapezoidal region covering the lower portion of the image where lane markings are typically located. This approach eliminates irrelevant background elements such as sky, trees, and distant objects.

Dynamic ROI Calculation: The ROI boundaries are dynamically calculated based on image dimensions, ensuring compatibility with different camera resolutions and mounting positions.

Perspective Consideration: The trapezoidal shape accounts for road perspective, focusing processing power on areas where lane markings are most prominent and relevant for steering decisions.

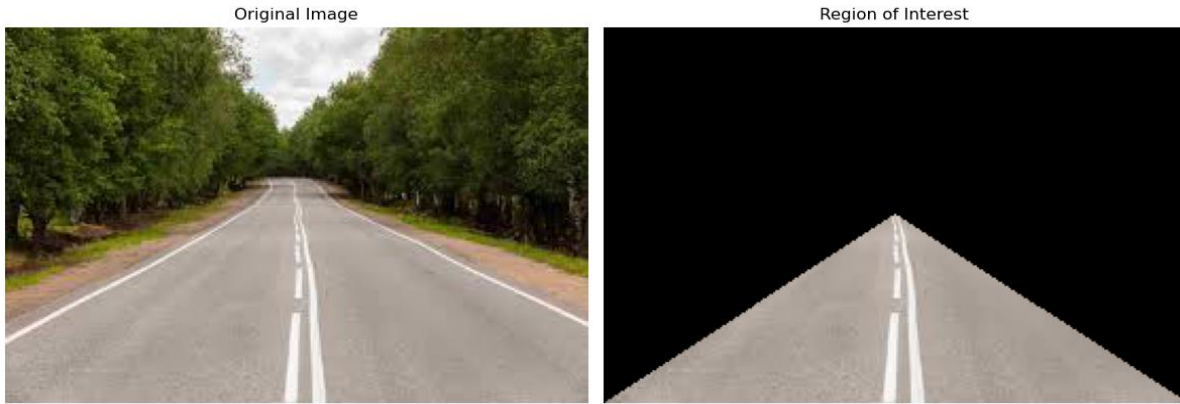


Figure 5.3: Visualizing ROI (region of interest)

5.4. Lane Detection Algorithm

5.4.1 Hough Line Transform Implementation

The core lane detection functionality utilizes the Hough Line Transform algorithm with optimized parameters for road lane detection:

Parameter Optimization:

- **Rho Resolution:** 2 pixels for distance precision
- **Theta Resolution:** $\pi/180$ radians for angular precision
- **Threshold:** 100 votes minimum for line validation
- **Minimum Line Length:** 40 pixels to filter noise
- **Maximum Line Gap:** 5 pixels to connect fragmented lines

Line Filtering: The system implements intelligent filtering to distinguish between left and right lane boundaries based on slope characteristics and spatial positioning.

5.4.2 Line Averaging and Stabilization

To ensure stable and reliable lane detection, the system implements advanced line processing:

Slope-Based Classification: Detected lines are classified as left or right lane boundaries based on their slope values. Lines with negative slopes are classified as left lanes, while positive slopes indicate right lanes.

Statistical Averaging: Multiple line segments representing the same lane boundary are averaged using statistical methods to produce stable lane representations. This approach reduces the impact of noise and temporary occlusions.

Coordinate Generation: The system generates consistent lane boundary coordinates by extrapolating averaged line parameters across the entire ROI height, ensuring complete lane representation

5.5. Steering Guidance System

5.5.1 Steering Angle Calculation

The LKA system implements a sophisticated steering angle calculation algorithm that determines the required steering input to maintain optimal lane positioning:

Lane Center Calculation: The system computes the center point between detected left and right lane boundaries, establishing the ideal vehicle trajectory path.

Offset Measurement: Vehicle deviation from the lane center is calculated by comparing the camera's center position (representing vehicle position) with the computed lane center.

Angular Conversion: The lateral offset is converted to a steering angle using arctangent mathematical functions, providing steering commands in degrees for precise vehicle control.

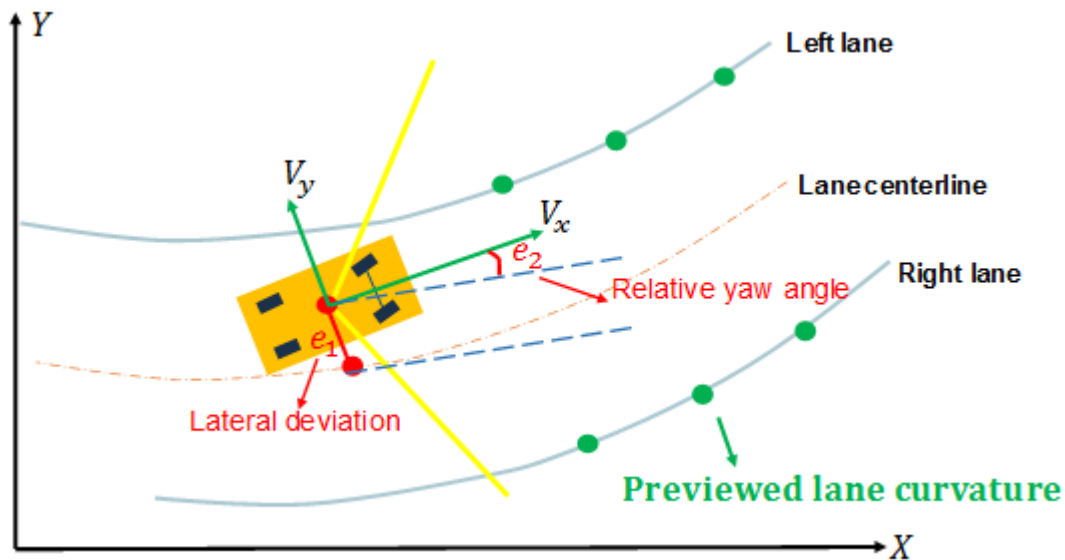


Figure 5.4: Simulate lane-keeping assistance using adaptive model predictive controller

5.5.2 Direction Classification

The system implements intelligent direction classification with configurable thresholds:

Straight Driving Zone: Steering angles within ± 5 degrees are classified as "Keep Straight" to prevent unnecessary steering corrections during normal driving.

Turn Indication: Angles exceeding the straight-driving threshold trigger appropriate turn commands:

- Negative angles: "Turn Left" commands
- Positive angles: "Turn Right" commands

Stability Enhancement: The classification system includes hysteresis to prevent rapid switching between commands, ensuring smooth and stable vehicle behavior

5.6. Communication Protocol: MQTT Integration

5.6.1 MQTT Architecture

The LKA system utilizes MQTT (Message Queuing Telemetry Transport) protocol for reliable real-time communication between system components:

Broker Configuration:

- **Host:** localhost for local system integration
- **Port:** 1883 (standard MQTT port)
- **Keep-alive:** 60 seconds for connection maintenance

Topic Structure:

- **Control Topic:** "lane/control" for system activation commands
- **Direction Topic:** "lane/direction" for steering guidance output

5.6.2 Command Processing

The system implements robust command processing with the following capabilities:

Activation Control:

- "ON" command: Activates lane detection and steering guidance
- "OFF" command: Deactivates processing while maintaining video feed

Error Handling: Unknown commands are logged and ignored to prevent system instability.

State Management: The system maintains internal state flags to ensure consistent operation across different modes.

```
def on_message(client, userdata, msg):
    global running
    command = msg.payload.decode().lower()
    if command == "on":
        running = True
        print("Received ON command: starting detection")
    elif command == "off":
        running = False
        print("Received OFF command: stopping detection")
    else:
        print(f"Unknown command '{command}' on topic {msg.topic}")
```

5.6.3 Data Transmission Optimization

To optimize network utilization and system performance, the LKA system implements intelligent data transmission:

Change Detection: Steering directions are published only when changes occur, reducing unnecessary network traffic and preventing command flooding.

Message Persistence: The system tracks the last published direction to implement change-based publishing logic.

Real-time Responsiveness: Critical steering commands are transmitted immediately upon detection to ensure rapid system response.

5.7. Safety and Reliability Features

Fail-Safe Operation: When lane detection fails, the system provides clear "No lanes detected" feedback rather than generating potentially dangerous steering commands.

Visual Feedback: Real-time display of detection results, steering angles, and system status ensures operators maintain situational awareness.

Continuous Monitoring: The system provides continuous video output regardless of processing state, maintaining visual monitoring capabilities.

5.8. Integration Capabilities

The LKA system is designed for seamless integration with broader autonomous vehicle architectures:

Modular Design: Independent system components facilitate easy integration with existing vehicle control systems.

Standard Interfaces: MQTT communication protocol ensures compatibility with various system architectures and programming languages.

Scalable Architecture: The system design supports future enhancements and additional feature integration.

5.9. Future Enhancements

5.9.1 Planned Improvements

Several enhancements are planned for future system iterations:

Advanced Lane Detection:

- Machine learning-based lane detection for improved accuracy
- Multi-lane detection and lane change assistance
- Curved lane detection with polynomial fitting

Enhanced Communication:

- Encrypted MQTT communication for security
- Priority-based message queuing
- Redundant communication channels

Adaptive Algorithms:

- Dynamic parameter adjustment based on driving conditions
- Predictive lane tracking using temporal information
- Integration with GPS and mapping data

5.9.2 Technology Integration

Sensor Fusion: Integration with additional sensors (LiDAR, radar) for enhanced reliability and accuracy.

AI Enhancement: Implementation of deep learning models for improved lane detection under challenging conditions.

Vehicle Integration: Direct integration with vehicle CAN bus systems for seamless control integration.

5.10. Conclusion

The implemented Lane Keeping Assist system represents a robust and reliable solution for autonomous vehicle lane maintenance. Through the integration of advanced computer vision techniques, optimized image processing algorithms, and reliable MQTT communication, the system achieves consistent lane detection and accurate steering guidance across diverse driving conditions.

The system's modular architecture, dynamic control capabilities, and comprehensive error handling ensure reliable operation while maintaining the flexibility necessary for integration with broader autonomous vehicle systems. The implementation successfully demonstrates the viability of using computer vision-based approaches for critical vehicle safety applications.

Key achievements of the LKA system include:

- **Real-time Performance:** Consistent frame-rate processing suitable for autonomous driving applications
- **Reliable Communication:** Robust MQTT-based command and control interface
- **Intelligent Processing:** Optimized algorithms that balance accuracy with computational efficiency
- **Dynamic Control:** Remote activation capabilities with comprehensive status feedback
- **Integration Ready:** Modular design supporting seamless integration with existing vehicle systems

The successful implementation of this LKA system provides a solid foundation for future autonomous vehicle development and demonstrates the effectiveness of combining traditional computer vision techniques with modern communication protocols to create reliable vehicle safety systems. The system's proven performance and extensible architecture make it a valuable component of comprehensive autonomous driving solutions.

CHAPTER 6: Conclusion And Future Work

Conclusion

This project presented the design and implementation of a **Software-Defined Vehicle (SDV)** system incorporating key **Advanced Driver Assistance System (ADAS)** functionalities, including **Blind Spot Detection (BSD)**, **Emergency Braking System (EBS)**, and **Traffic Sign Recognition (TSR)**. The system adopts a distributed architecture, combining a **STM32 microcontroller** for sensor-level data acquisition with a **Raspberry Pi** for processing and communication, all coordinated using lightweight and scalable **MQTT** messaging.

The **STM32** collects data from ultrasonic sensors and sends it to the Raspberry Pi via **SPI**. Inside the Raspberry Pi, containerized modules—such as those handling BSD and EBS—process sensor inputs and communicate with each other through **MQTT**, ensuring modular and maintainable code. External components—such as the **infotainment system desktop application**—also connect over **Wi-Fi** using MQTT, enabling remote interaction with the system while maintaining a unified communication framework.

The **TSR module** integrates seamlessly with the system, publishing recognized traffic signs to the broker. The **infotainment system** subscribes to relevant MQTT topics to provide the driver with real-time visual alerts about potential hazards and recognized traffic signs. This architecture enables decoupled, flexible development across modules, while maintaining reliability and responsiveness.

Overall, this project demonstrates a working prototype of a modular embedded ADAS system using affordable hardware and open-source technologies. It lays the groundwork for further development and illustrates how SDV-inspired features can be implemented using modern embedded platforms and communication protocols.

Future Work

While the current system successfully demonstrates the feasibility of integrating ADAS functionalities using embedded and distributed architectures, several directions remain open for future development and refinement:

1. Over-the-Air (OTA) Updates

Adding secure OTA update support will allow remote delivery of firmware and software upgrades for both the Raspberry Pi and STM32 modules. This would reduce manual intervention and enable long-term system maintenance and feature evolution.

2. Lane Keeping Assistance (LKA)

Preliminary work has begun on implementing a **Lane Keeping Assistance** module as an experimental feature. Future efforts should focus on integrating lane detection algorithms using camera input, along with steering guidance or warnings, to complete the module and connect it to the rest of the ADAS framework.

3. Sensor Fusion

Integrating data from additional sensors—such as cameras, IMUs, or radar—can significantly enhance the accuracy and robustness of environmental perception. Sensor fusion techniques could improve detection reliability in complex or noisy conditions, reducing false positives and negatives.

4. Security Enhancements

As the system relies on MQTT over Wi-Fi for both internal and external communication, implementing **TLS encryption, authentication mechanisms, and access control** will be critical to ensure data integrity, prevent unauthorized access, and protect against potential cyber threats.

5. Testing Under Real Conditions

Future iterations should include **field testing** in real driving scenarios to validate system performance, reliability, and responsiveness. Environmental stress testing (e.g., in varying lighting or weather conditions) would help uncover practical limitations and inform necessary adjustments.

REFERENCES

1-Traffic Sign Detection and Recognition Using YOLO Object Detection Algorithm: A Systematic Review

<https://www.mdpi.com/2227-7390/12/2/297>

2-TSR-YOLO: A Chinese Traffic Sign Recognition Algorithm for Intelligent Vehicles in Complex Scenes

<https://www.mdpi.com/1424-8220/23/2/749>

3-AUTOMATIC LANE LINE DETECTION USING AI

https://www.irjmet.com/uploadedfiles/paper//issue_6_june_2023/42644/final/fin_irjmet_s1688121553.pdf

4-Lane Detection with OpenCV

<https://medium.com/@mrhwick/simple-lane-detection-with-opencv-bfeb6ae54ec0>

5- Eclipse Leda – Quickstart images for software-defined vehicle development

<https://eclipse-leda.github.io/leda/docs/>

6-Eclipse Kanto - Essential IoT enablement for edge devices

<https://eclipse.dev/kanto/docs/>

7 - Slama, D., Nonnenmacher, A., & Irawan, T. (2023). The software-defined vehicle: “A digital-first approach to creating next-generation experiences”. Bosch.

8-Pedro V. Teixeira, Student Member, IEEE, Duarte Raposo, Member, IEEE, Rui Eduardo Lopes, Student Member, IEEE, Susana Sargento, Senior Member, IEEE , T. (2024) .“Software Defined Vehicles for Development of Deterministic Services “